
Sage: An Offline-First Academic Assistant Using RAG with Tool-Integrated Workflows

Project Team

Ahmad Raza (SOFTF22BSR04)
ahmad.raza@scholar.tu.edu.pk

Abdullah Khan (SOFTF22BSR28)
abdullah.khan@scholar.tu.edu.pk

Supervisor: Not Assigned



Session 2022–2026
Bachelor of Science in Software Engineering

Department of Computer Science & Information Technology
Thal University Bhakkar

Date: June 2026

Acknowledgements

First and foremost, the authors bow in humble gratitude before Allah Almighty, the Most Gracious and the Most Merciful, for granting the wisdom, health, strength, and resolve to successfully complete this project. We also wish to extend our sincere appreciation to our families, friends, and peers, whose encouragement, constructive suggestions, and constant moral support served as a continuous source of motivation throughout this journey. Finally, we acknowledge the open-source communities behind LangChain, LangGraph, llama.cpp, and the broader ecosystem of tools upon which Sage is built.

Contents

List of Abbreviations	7
Chapter 1: Introduction	9
1.1 Background	9
1.2 Problem Statement	9
1.3 Motivation	9
1.4 Project Objectives	10
1.5 Scope and Boundaries	10
1.6 Report Organisation	11
Chapter 2: Literature Review	11
2.1 Overview of the Domain	11
2.2 Related Work and Existing Systems	12
2.3 Identified Gaps	13
2.4 Comparative Analysis	13
Chapter 3: Methodology	14
3.1 Development Methodology	14
3.2 Technology Selection Rationale	15
3.3 Data Collection and Preprocessing	16
3.4 Development Timeline and Milestones	17
Chapter 4: Requirements Analysis	17
4.1 Stakeholder Analysis	17
4.2 Functional Requirements	18
4.3 Non-Functional Requirements	19
4.4 Use Case Descriptions	19
4.4.1 Ask an Explain Question	19
4.4.2 Generate and Take a Quiz	19
4.4.3 Conduct Multi-Source Research	20
4.4.4 Generate a Study Plan	20
4.4.5 Generate a Diagram	21
4.4.6 Diagnose and Fix Code	21
4.4.7 Extended Reasoning (Thinking Mode)	22
4.4.8 General Conversation	22
4.5 Requirements Traceability Matrix	23
Chapter 5: System Design	24
5.1 System Architecture Overview	24
5.2 Component Breakdown	25
Router (agents/router.py)	25
Retrieval Node (agents/retrieval.py)	25
Hybrid Retrieval Pipeline (rag/retrieval.py)	25
Agent Nodes	25
LLM Lifecycle Manager (llm.py)	26
Semantic Memory (memory.py)	26
Network Monitor (network.py)	26
5.3 Data Flow and Processing Pipeline	26
Document Ingestion Flow	26
Query Answering Flow	27
Agent State Machine	27
5.4 Database / Storage Design	27
SQLite Database	27

Vector Store (ChromaDB)	28
BM25 Sparse Index	28
5.5 Interface Design	28
5.6 Security and Privacy Design	29
5.6.1 Privacy-Preserving Architecture	29
5.6.2 Sandboxed Code Execution	30
5.6.3 API Security	30
5.6.4 Supply Chain Security	30
5.7 Deployment Architecture	30
5.7.1 Build Pipeline	30
5.7.2 Installer Tiers	31
5.7.3 Continuous Integration and Deployment	31
6.1 Development Environment and Tools	31
6.2 Key Implementation Details	32
6.2.1 Deterministic Router Design	32
6.2.2 Reciprocal Rank Fusion Implementation	32
6.2.3 Hierarchical Model Strategy	32
6.2.4 Error Boundary Pattern	32
6.2.5 Windows Pipe Buffer Deadlock Prevention	33
6.3 Challenges and Solutions	33
Challenge 1: Context Window Exhaustion on Low-RAM Machines	33
Challenge 2: BM25 and Dense Score Incompatibility	33
Challenge 3: LLM JSON Parsing Failures	33
6.4 User Interface Implementation	34
6.4.1 Chat-Centric Layout	34
6.4.2 Mode Selection and Agent Routing	34
6.4.3 Session Management	34
6.4.4 Streaming and Progressive Rendering	34
6.4.5 Artefact Downloads	34
6.5 Additional Challenges and Solutions	35
Challenge 4: Mermaid Diagram Syntax Validation	35
Challenge 5: Native Desktop Integration on Windows	35
Chapter 7: Testing and Evaluation	35
7.1 Testing Strategy	35
7.2 Unit and Integration Tests	36
7.3 System Testing	37
7.4 Results and Discussion	37
7.4.1 Quantitative Evaluation Results	37
7.4.2 Analysis and Discussion	38
7.5 Code Coverage Analysis	38
7.6 Continuous Integration and Quality Assurance	39
Chapter 8: Cross-Departmental Adaptability and Institutional Adoption	39
8.1 Curriculum-Agnostic Design Philosophy	39
8.2 Institutional Configuration and Branding	40
8.3 Cross-Departmental Deployment Pipeline	40
8.4 Scalability Considerations	41
Chapter 9: Conclusion and Future Work	41
9.1 Summary of Contributions	41
9.2 Limitations	42
9.3 Future Work	42
9.4 Societal Impact and Significance	42

9.5 Lessons Learned	43
References	44
Appendix A: Installation and Quick Start Guide	45
Appendix B: Application Screenshots	46

List of Figures

1	Use case diagram for Sage	21
2	Document ingestion sequence from file upload to vector and sparse index storage	27
3	Query answering flow through the agent pipeline	27
4	LangGraph agent state machine	28
5	UI navigation flow	29
6	First run experience	46
7	Home screen: the main application interface during active use	46
8	Quiz mode: structured quiz with multiple question types generated from course material . . .	47
9	Research mode: multi-source research with citations and downloadable PDF export	47
10	Explain mode: curriculum-grounded explanation with citation tags	47
11	Diagram mode: system architecture or algorithm visualization rendered as SVG	48
12	Code Fix mode: code diagnosis, automatic repair, and sandboxed execution output	48

List of Tables

1	Comparative Analysis of Academic Assistants	13
2	Development Timeline and Milestones	17
3	Requirements Traceability Matrix	23

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BM25	Best Matching 25 (probabilistic retrieval function)
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
ER	Entity-Relationship
FTS	Full-Text Search
GGUF	GPT-Generated Unified Format
GPU	Graphics Processing Unit
HEC	Higher Education Commission (Pakistan)
KU	Knowledge Unit
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	Normalised Discounted Cumulative Gain
NSIS	Nullsoft Scriptable Install System
ONNX	Open Neural Network Exchange
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SPA	Single-Page Application
SSE	Server-Sent Events
TOML	Tom's Obvious Minimal Language
TPS	Tokens Per Second
VRAM	Video Random Access Memory
WAL	Write-Ahead Logging

Abstract

AI-powered study tools have transformed how students engage with academic material, yet the dominant deployment model, cloud-hosted large language model inference remains inaccessible to many: it demands persistent internet connectivity, incurs recurring subscription costs, and transmits sensitive course materials to third-party servers. These constraints fall disproportionately on students at resource-constrained institutions in developing regions, where network infrastructure is inconsistent and data sovereignty is a genuine concern.

This project presents **Sage**, a privacy-preserving, offline-first academic assistant for undergraduate computer science and engineering students. Sage runs quantized large language models entirely on consumer hardware via llama.cpp (GGUF format), requiring no API keys, cloud accounts, or internet access for its core functionality. The system is delivered as a native Windows desktop application with a React/TypeScript frontend, a FastAPI backend, and a pywebview shell, a self-contained package that manages the full lifecycle of local LLM servers, vector stores, and retrieval indices.

Sage’s core technical contribution is a multi-agent orchestration architecture built on LangGraph, in which a deterministic router dispatches queries to one of eight specialised agent nodes: curriculum-grounded explanation with citation formatting, quiz generation and automated evaluation, Mermaid diagram synthesis with iterative syntax validation, structured day-by-day study plan generation with spaced repetition, multi-source research with PDF export, sandboxed code diagnosis and repair, extended chain-of-thought reasoning, and general-purpose conversation with persistent semantic memory. The retrieval subsystem implements a hybrid pipeline that fuses dense vector search (ChromaDB with BGE-small-en-v1.5 embeddings) and BM25 sparse retrieval through Reciprocal Rank Fusion, grounding all agent outputs in the student’s own ingested course material. A hierarchical two-model strategy, a primary Qwen2.5 generation model (2B on CPU, 4B on CUDA) paired with a lightweight 0.8B utility model for background tasks, balances generation quality against resource efficiency on low-end hardware.

Informal evaluation across CPU-only and GPU-accelerated configurations confirms functional correctness across all eight agent modes, response latencies suitable for interactive academic use (3–15 seconds for explain-mode queries), and reliable offline operation. The project demonstrates that a richly featured, curriculum-aware academic assistant can be deployed entirely on consumer hardware, making AI-assisted learning accessible regardless of infrastructure constraints.

Chapter 1: Introduction

1.1 Background

The intersection of natural language processing and education has accelerated considerably with the proliferation of large language models. Tools such as ChatGPT, Gemini, and Claude have demonstrated that conversational AI can assist with explaining concepts, answering questions, and generating structured academic content. The global educational technology market has responded to these capabilities with widespread integration of AI-powered study tools across institutions worldwide. However, the dominant deployment model for these systems relies on cloud-hosted inference, which requires persistent internet connectivity, incurs per-query costs, and transmits potentially sensitive academic data, including personal study materials and examination content, to third-party servers [13].

These constraints are particularly acute in developing regions, where university students frequently contend with intermittent electricity, bandwidth limitations, and prohibitive costs for subscription-based services. In Pakistan alone, numerous public-sector universities located in semi-urban and rural districts lack the robust network infrastructure that cloud-dependent AI tools presuppose. Students at these institutions are effectively excluded from the benefits of AI-assisted learning, not due to a lack of capable hardware, but because existing tools are architected around assumptions of persistent, high-bandwidth connectivity.

Concurrently, advances in model quantization (notably the GGUF format used by llama.cpp [2]) and the release of capable small language models in the 0.5B to 4B parameter range [5] have made it feasible to run inference on consumer-grade hardware. The GGUF format represents model weights in 4-bit or 8-bit precision, reducing memory requirements by up to 75 percent relative to full-precision models with minimal quality degradation, as empirically validated by recent quantization studies [12]. Projects such as Ollama and llama.cpp have lowered the barrier to local LLM deployment, while frameworks such as LangChain and LangGraph [6] provide the orchestration abstractions needed to build multi-step agent workflows around these models. These developments create an opportunity to build academic tools that operate entirely on-device, preserving privacy and eliminating infrastructure costs.

The convergence of these two trends, the growing demand for AI-assisted academic tools and the increasing feasibility of local inference on consumer hardware, defines the research space in which this project operates. Sage was conceived at this intersection, with the explicit goal of demonstrating that a richly featured, curriculum-grounded academic assistant can be delivered as a self-contained desktop application that requires no internet access, no API keys, and no cloud accounts for its core functionality.

1.2 Problem Statement

Students in academic institutions face several challenges when using AI-powered study tools. First, cloud-based assistants require continuous internet access, which is unreliable or expensive in many institutional and geographic contexts particularly in developing regions where network infrastructure is inconsistent. Second, uploading course materials (lecture slides, textbooks, past papers) to external servers raises legitimate privacy and intellectual property concerns, especially when materials are proprietary or contain unpublished examination content. Third, general-purpose chatbots lack integration with the student’s own curriculum; they cannot retrieve specific information from ingested course materials, generate quizzes grounded in syllabus content, or produce study plans aligned with a particular programme’s structure.

Furthermore, existing offline-capable tools such as PrivateGPT [14] and AnythingLLM [15] focus primarily on document question-answering and lack the specialised academic workflows, quiz generation, diagram synthesis, structured study planning, multi-source research with citation tracking, that a student-facing tool requires. There is no integrated, offline-first desktop application that combines retrieval-augmented generation [1] with a suite of tool-augmented academic agent workflows.

1.3 Motivation

The motivation for Sage arises from direct experience with the constraints described above. Students at institutions such as Thal University Bhakkar frequently contend with unreliable internet connectivity, limited

budgets for subscription-based AI services, and a lack of specialised academic tools that integrate with their course materials. A locally-deployed assistant that processes queries against ingested lecture content, generates quizzes for self-assessment, renders technical diagrams, and produces structured study plans, all without requiring an internet connection or API key, directly addresses these pain points.

Beyond the immediate practical utility, the project is motivated by the principle that AI-assisted learning tools should be accessible regardless of infrastructure constraints. An offline-first architecture ensures that the tool remains functional during network outages, in examination preparation environments where internet access is restricted, and in contexts where data sovereignty is a concern.

1.4 Project Objectives

The project objectives are as follows:

1. To design and implement a multi-agent orchestration framework using LangGraph that routes user queries to specialised agent nodes based on intent classification.
2. To implement a hybrid retrieval-augmented generation pipeline combining dense vector search (ChromaDB) and sparse retrieval (BM25) with Reciprocal Rank Fusion for grounding responses in ingested curriculum material.
3. To deploy quantized large language models locally via llama.cpp with automatic hardware detection and context window scaling for both CPU and CUDA backends.
4. To implement eight specialised agent workflows: explanation, quiz generation and evaluation, diagram synthesis, study plan generation, multi-source research, code diagnosis, extended reasoning, and general conversation.
5. To develop a long-term semantic memory system that extracts, deduplicates, and retrieves student-specific facts across sessions.
6. To deliver the system as a native Windows desktop application with a React/TypeScript frontend, FastAPI backend, and pywebview shell.
7. To validate the system through a comprehensive test suite achieving 85 percent or greater code coverage.
8. To design the system with a curriculum-agnostic architecture that enables zero-configuration cross-departmental adoption across diverse academic programmes.

1.5 Scope and Boundaries

Defining a clear project scope is essential for establishing realistic expectations and evaluating the system against its intended purpose. This section delineates what falls within the scope of the project and what has been deliberately excluded.

In Scope:

- Design and implementation of a complete offline-first desktop application for Windows 10 and Windows 11.
- Local deployment of quantized large language models (Qwen3.5 family) via llama.cpp in GGUF format, supporting both CPU-only and NVIDIA CUDA-accelerated inference.
- A multi-agent orchestration framework with eight distinct agent nodes, each addressing a specific academic workflow.
- A hybrid retrieval-augmented generation pipeline that fuses dense vector search and BM25 sparse retrieval to ground responses in user-ingested course materials.
- Support for ingesting academic documents in PDF, DOCX, PPTX, Markdown, and plain text formats.
- A persistent semantic memory system that extracts and recalls student-specific facts across sessions.
- A self-contained NSIS installer that packages all dependencies (portable Python runtime, pre-compiled server binaries, model files, frontend build) into a single distributable executable.
- A continuous integration and continuous deployment pipeline using GitHub Actions for automated linting, testing, building, and release management.
- A curriculum-agnostic configuration system that enables adaptation to any academic department or institution without source code modification.

Out of Scope:

- Multi-platform deployment (macOS and Linux). The current implementation targets Windows exclusively due to the pywebview/WebView2 dependency and the NSIS installer toolchain.
- Multi-user or server-based deployment. Sage is designed as a single-user desktop application; collaborative features and concurrent multi-user access are not addressed.
- Multi-modal document understanding. The ingestion pipeline extracts text content from documents but does not process images, charts, or complex table structures embedded within academic materials.
- Fine-tuning or training of language models. The system uses pre-trained, quantized models without modification; no model training or fine-tuning is performed.
- Formal user studies with statistically significant sample sizes. System validation is performed through automated testing and informal exploratory evaluation rather than controlled experiments with human participants.
- Real-time collaborative editing or cloud synchronisation of study materials.

This scope definition ensures that the project remains focused on its core contribution: demonstrating that a richly featured, privacy-preserving academic assistant can be deployed entirely on consumer hardware, while acknowledging the boundaries that future work may extend.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature on retrieval-augmented generation, local LLM inference, and existing academic assistant tools, identifying the research gaps that Sage addresses. Chapter 3 describes the development methodology, technology selection rationale, data collection procedures, and project timeline. Chapter 4 presents the requirements analysis, including stakeholder identification, functional and non-functional requirements, use case descriptions, and a requirements traceability matrix. Chapter 5 describes the system design in detail, covering the five-layer architecture, component breakdown, data flow pipelines, storage schema, security and privacy design, deployment architecture, and interface design. Chapter 6 discusses the implementation, highlighting key technical decisions, challenges encountered during development, and user interface construction. Chapter 7 details the testing strategy, presents evaluation results, and discusses continuous integration practices. Chapter 8 addresses the cross-departmental adaptability of the system and its institutional adoption potential. Chapter 9 concludes the report with a summary of contributions, an honest assessment of limitations, societal impact considerations, lessons learned, and directions for future work.

Chapter 2: Literature Review

2.1 Overview of the Domain

The domain of AI-assisted academic tools sits at the intersection of information retrieval, natural language generation, and educational technology. The evolution of information retrieval from keyword-based methods to dense neural retrieval has enabled systems to match queries to documents based on semantic similarity rather than lexical overlap. BM25 remains the most widely used sparse retrieval function: Robertson and Zaragoza [4] provide a comprehensive treatment of the probabilistic relevance framework underlying BM25 and its variants, demonstrating its robustness across diverse query types. Dense retrieval, by contrast, maps both queries and documents into continuous vector spaces using learned transformer embeddings, excelling at capturing paraphrased or conceptually related content [8]. Contemporary retrieval systems increasingly combine both paradigms in hybrid configurations that outperform either method in isolation; empirical studies have shown that fusing dense and sparse results through rank-based methods can yield substantial recall improvements over dense-only retrieval across diverse query types [3].

Retrieval-Augmented Generation (RAG) directly addresses the knowledge currency and hallucination problems of large language models (LLMs): their knowledge is frozen at training time, and they are prone to generating plausible but factually incorrect content when queried on specific documents they have not seen. Lewis et al. [1] introduced the seminal RAG formulation, demonstrating that augmenting generation with non-parametric

retrieval from an external knowledge base significantly improves factual accuracy on knowledge-intensive tasks. Gao et al. [10] provide a comprehensive survey of RAG paradigms, from naïve single-pass retrieval to modular and advanced configurations, detailing how retrieval, generation, and augmentation components have co-evolved and identifying open challenges around retrieval quality, context faithfulness, and latency.

Large language models based on the transformer architecture [11] have demonstrated broad capabilities in text generation, summarisation, question answering, and structured output production. The release of capable open-weight model families in the sub-10B parameter range, including Qwen, Llama, Phi, and Gemma, has made it possible to run competitive language models on consumer hardware. The Qwen2.5 technical report [5] establishes that models as small as 0.5B–4B parameters, trained on high-quality data with post-training alignment, can perform competitively with much larger predecessors on reasoning, coding, and instruction-following benchmarks. Quantisation techniques, particularly the GGUF format supported by llama.cpp [2], reduce model memory footprint by representing weights in 4-bit or 8-bit precision with minimal quality degradation [12], enabling inference on machines with 8–16 GB of RAM and making local deployment feasible for students on consumer-grade hardware.

Agent frameworks such as LangGraph [6] extend the RAG paradigm by modelling multi-step workflows as stateful directed graphs. Each node in the graph is a Python function powered by an LLM or tool; edges define conditional routing; and state is checkpointed persistently after every node execution, making the system resilient to failures. This architecture enables the kinds of complex, multi-turn academic workflows e.g., iterative quiz evaluation, diagram retry loops, multi-source research synthesis, that go beyond what a single prompt-response cycle can achieve.

The BGE embedding model family [7], developed by the Beijing Academy of Artificial Intelligence (BAAI), achieves top-tier performance on the MTEB benchmark for dense retrieval. The BGE-small-en-v1.5 variant used in Sage produces 384-dimensional sentence embeddings at inference speeds suitable for on-device use in ONNX format via FastEmbed, providing a lightweight yet effective dense retrieval component without GPU requirements.

2.2 Related Work and Existing Systems

Several existing systems address aspects of the problem space that Sage targets, though none provides the same combination of offline operation, multi-agent workflows, and curriculum-integrated retrieval.

General-purpose AI chatbots including ChatGPT, Gemini, and Claude offer high-quality conversational assistance with academic topics, benefiting from large model sizes and extensive fine-tuning. However, they cannot be grounded in a student’s specific course materials, require paid subscriptions for sustained use, and transmit all queries, including potentially sensitive examination content, to external servers. Liu et al. [13] identify data privacy, recurring costs, and internet dependency as the dominant barriers to sustained AI-assisted learning adoption in institutional settings, findings that directly motivate the offline-first design of Sage.

Local RAG tools such as PrivateGPT [14] and AnythingLLM [15] enable users to run document question-answering pipelines on their own hardware. PrivateGPT, released in May 2023, was among the first projects to combine a local LLM backend (via llama.cpp) with a local vector store and document Q&A interface, establishing the feasibility of privacy-preserving offline document assistants [14]. AnythingLLM extends this pattern with a browser-based interface, multi-user support, and agent-like tool invocations, supporting connections to both local LLM backends and cloud APIs [15]. However, both tools are designed as general-purpose document chat systems and lack the specialised academic workflows that student-facing use demands: neither provides automated quiz generation and evaluation, diagram synthesis, structured study planning with prerequisite ordering, or multi-source research with citation tracking. Sage differs by embedding these workflows as first-class agent nodes in a LangGraph state graph, rather than treating them as one-off prompt templates.

LLM-based intelligent tutoring systems represent an active research direction [11]. Liu et al. [11] present LPITutor, a RAG-augmented personalised tutoring system that uses adaptive prompt engineering to dynamically adjust responses to a learner’s prior knowledge and progress. Their evaluation demonstrates

that RAG-augmented systems produce measurably more accurate and pedagogically effective responses than static prompt-only systems by grounding outputs in vetted educational materials, a finding that validates the retrieval-augmented approach at the core of Sage. However, LPITutor is a cloud-hosted web system without offline capability, and it does not provide structured quiz generation, diagram synthesis, or study planning features.

Automated question generation from educational text is a well-studied problem in NLP [16]. Recent work by Hasan et al. [16] demonstrates that fine-tuned LLMs can generate diverse, high-quality question sets (MCQ, conceptual, factual) from course material using prompt engineering, substantially reducing the manual effort required for assessment preparation. Elkins et al. [17] evaluate the educational utility of LLM-generated questions and find that, with appropriate prompting, GPT-family models can produce questions that are rated as useful by human educators, though quality varies with topic complexity Sage’s quiz agent builds on these findings by combining hybrid retrieval, ensuring questions are grounded in specific course chunks rather than the model’s parametric knowledge with a structured JSON generation schema that enforces question type diversity.

2.3 Identified Gaps

The review of existing systems reveals a consistent pattern: tools that provide high-quality AI assistance require cloud connectivity and incur ongoing costs, while tools that operate locally are limited to basic document question-answering without the specialised workflows that academic use demands. No existing system combines all of the following capabilities in a single, offline-first application:

A hybrid retrieval pipeline that merges dense and sparse search results to ground responses in specific course material. A multi-agent architecture where different query types are dispatched to specialised processing nodes rather than handled by a single monolithic prompt. A tool-augmented workflow that integrates sandboxed code execution, diagram rendering, search engines, and document export as first-class agent capabilities. A long-term memory system that learns student-specific facts across sessions and injects them into future interactions. A hierarchical model strategy that offloads auxiliary tasks (memory extraction, history compression) to a lightweight utility model, preserving the primary model’s context window for generation. A native desktop application that manages the full lifecycle of local LLM servers, vector stores, and the frontend, without requiring the user to configure any infrastructure.

Sage was designed to fill this gap by integrating these capabilities into a cohesive system that runs entirely on consumer hardware without external dependencies.

2.4 Comparative Analysis

To position Sage within the landscape of existing tools, Table 1 presents a systematic comparison across key capability dimensions. Each tool is evaluated on whether it supports a given feature natively, partially, or not at all.

Table 1: Comparative Analysis of Academic Assistants

Capability	ChatGPT / Gemini	PrivateGPT [14]	AnythingLLM [15]	LPITutor [11]	Sage
Offline operation	No	Yes	Partial	No	Yes
Local LLM inference	No	Yes	Partial	No	Yes
Hybrid retrieval (dense + sparse)	No	Dense only	Dense only	Dense only	Yes (RRF)
Curriculum-scoped retrieval	No	No	No	Partial	Yes

Capability	ChatGPT / Gemini	PrivateGPT [14]	AnythingLLM [15]	LPITutor [11]	Sage
Multi-agent orchestration	No	No	Partial	No	Yes (LangGraph)
Quiz generation and evaluation	No	No	No	No	Yes
Diagram synthesis (Mermaid)	No	No	No	No	Yes
Structured study planning	No	No	No	No	Yes
Multi-source research with export	Partial	No	No	No	Yes
Code diagnosis and repair	No	No	No	No	Yes
Semantic memory (cross-session)	Limited	No	No	No	Yes
Data privacy (on-device)	No	Yes	Partial	No	Yes
Self-contained installer	N/A	No	No	N/A	Yes (NSIS)
Cross- departmental adaptability	N/A	No	No	No	Yes
Cost to end user	Subscription	Free	Free/Paid	N/A	Free

Table 1 demonstrates that no existing tool provides the combination of offline-first operation, multi-agent academic workflows, hybrid curriculum-grounded retrieval, and native desktop packaging that Sage delivers. Cloud-based tools such as ChatGPT and Gemini excel in generation quality due to large model sizes, but they fundamentally cannot operate offline, cannot be grounded in user-specific course materials, and impose recurring costs. Local RAG tools such as PrivateGPT and AnythingLLM address the privacy and offline dimensions, but they are limited to document question-answering and do not provide the structured academic workflows (quiz generation, diagram synthesis, study planning, research with citations) that differentiate Sage. LPITutor demonstrates the pedagogical value of RAG-augmented tutoring, but it is a cloud-hosted system without offline capability or multi-agent specialisation.

The comparative analysis confirms that Sage occupies a unique position in this design space: it is the only system that integrates all of the identified capabilities into a single, self-contained, offline-first desktop application.

Chapter 3: Methodology

This chapter describes the development methodology adopted for the project, the rationale behind key technology selections, the data collection and preprocessing procedures used to build the curriculum knowledge base, and the project timeline from inception to completion.

3.1 Development Methodology

The development of Sage followed an **iterative and incremental** methodology, informed by Agile principles but adapted to the constraints of a two-person academic project team. The methodology was characterised

by short development cycles (typically one to two weeks), continuous integration from the earliest stages, and frequent re-evaluation of priorities based on emerging technical challenges.

The decision to adopt an iterative approach rather than a sequential waterfall model was driven by the experimental nature of the project. Many critical design parameters, such as the optimal context window size for low-RAM machines, the reliability of LLM-generated JSON, and the effectiveness of different retrieval fusion strategies, could only be determined through implementation and testing. A sequential approach would have required these parameters to be specified during the design phase, before sufficient empirical evidence was available.

The development process was organised into the following iterative phases spanning approximately 3.5 months:

Phase 1: Foundation and Feasibility. This phase focused on establishing the core technical feasibility of local LLM inference on consumer hardware, including evaluating quantized models and building the initial llama-server lifecycle manager.

Phase 2: Tool Development. This phase involved developing and testing the individual external tools required by the system, such as the sandboxed code execution environment, diagram rendering validations, and document export utilities.

Phase 3: Agentic Workflow Development. The LangGraph orchestration framework was implemented, and the specific agent nodes (e.g., explain, quiz, diagram, study plan) were constructed to handle specialised academic tasks.

Phase 4: Integration of Tools with Workflows. The previously developed tools were integrated into the agentic workflows, enabling agents to invoke search engines, execute code, and render diagrams dynamically within the state graph.

Phase 5: Memory and Persistence. A semantic memory subsystem was introduced to extract and store student-specific facts across sessions, alongside the SQLite-based LangGraph state checkpointing for conversation persistence.

Phase 6: Desktop Application and Packaging. The native desktop application shell (pywebview) and React frontend were developed and packaged into a distributable NSIS installer using a custom build orchestrator.

Phase 7: CI Workflows. Continuous integration pipelines were established to automate testing, linting, type checking, and architecture verification across multiple environments.

Phase 8: Testing, Evaluation, and Documentation. The final phase focused on system-level evaluation, bug fixing, test suite expansion, and the compilation of project documentation and reports.

Throughout all phases, version control was maintained using Git with a structured branching strategy (main, dev-main, and feature branches).

3.2 Technology Selection Rationale

The technology stack for Sage was selected based on a systematic evaluation of alternatives against the project’s core constraints: offline-first operation, consumer hardware compatibility, academic workflow specialisation, and single-installer deployment. This section documents the rationale for each major technology decision.

LLM Inference Engine: llama.cpp. The choice of llama.cpp as the inference backend was driven by three factors: (1) it supports the widest range of quantized model formats (GGUF), enabling aggressive memory reduction; (2) it provides both CPU and CUDA backends with automatic hardware detection; and (3) it exposes an OpenAI-compatible HTTP API via llama-server, allowing the Python application to interact with the model through standard HTTP requests rather than requiring C/C++ bindings. Alternatives considered included Ollama (higher-level but less configurable, with its own process management overhead), vLLM (GPU-only, server-oriented), and CTranslate2 (limited model support). llama.cpp was selected for its combination of broad hardware support, mature quantization ecosystem, and fine-grained control over inference parameters.

Language Model Family: Qwen3.5. The Qwen3.5 model family was selected based on the latest benchmark comparisons for official models’ performance and its feasibility for offline usage. It provides strong features such as a 256k context support window and a lower KV cache size, which are critical for processing extensive academic materials locally on consumer hardware. The availability of official GGUF quantizations and the model’s relatively compact memory footprint at Q4_K_M precision further supported the selection.

Orchestration Framework: LangGraph. LangGraph was chosen over alternative agent frameworks (AutoGen, CrewAI, custom state machines) because it provides (1) a typed state graph abstraction that maps naturally to the multi-agent routing pattern, (2) built-in state checkpointing via SQLite, enabling session persistence and fault recovery, and (3) tight integration with LangChain’s tool and prompt abstractions, reducing boilerplate. The graph-based topology also provides a clear visual representation of the system’s control flow, aiding both development and documentation.

Vector Store: ChromaDB. ChromaDB was selected for its lightweight, embedded deployment model (no external server process required), its Python-native API, and its support for persistent on-disk storage. Alternatives considered included FAISS (high performance but lacks metadata filtering and requires manual persistence), Qdrant (more feature-rich but requires a separate server process), and Weaviate (cloud-oriented). ChromaDB’s ability to run in-process within the Python application, with zero configuration, aligned with the self-contained deployment requirement.

Embedding Model: BGE-small-en-v1.5. The BGE-small-en-v1.5 model was selected for its balance of retrieval quality and inference efficiency. It produces 384-dimensional embeddings, which are sufficiently expressive for academic text retrieval while requiring significantly less memory and computation than larger embedding models (such as BGE-large, which produces 1024-dimensional vectors). The availability of a pre-converted ONNX variant via FastEmbed enables CPU-only inference without GPU dependencies.

Frontend: React with TypeScript. React 18 with TypeScript was chosen for the frontend due to its mature ecosystem, strong typing guarantees, and the availability of libraries for Markdown rendering, syntax highlighting, LaTeX math rendering, and Mermaid diagram display. The frontend is built with Vite for fast development iteration and bundled as static assets for distribution.

Desktop Shell: pywebview. pywebview was selected over Electron for the desktop shell because it renders web content using the system’s existing WebView2 runtime (Edge Chromium on Windows), eliminating the need to bundle an entire browser runtime. This reduces the installer size by approximately 100 to 150 MB compared to an Electron-based alternative. The trade-off is platform-specific behaviour (WebView2 is Windows-only), which is acceptable given the project’s Windows-only scope.

Backend: FastAPI. FastAPI was chosen for the backend API layer due to its native async support (critical for concurrent LLM and retrieval operations), automatic OpenAPI documentation generation, and Pydantic-based request/response validation. The combination of FastAPI with Uvicorn provides a production-grade ASGI server suitable for local deployment.

3.3 Data Collection and Preprocessing

The curriculum data that populates Sage’s retrieval indices was collected from publicly available academic materials recommended by the Higher Education Commission (HEC) of Pakistan for undergraduate Computer Science, Information Technology, and Software Engineering programmes. The data collection and preprocessing pipeline proceeded through the following stages:

Source Collection. Course materials were organised by programme code (BSCS, BSIT, BSSE), semester number (1 through 8), and course code. Supported file formats included PDF textbook chapters, PPTX lecture slides, DOCX handouts, Markdown notes, and plain text files. The raw materials were structured in a hierarchical directory layout that encodes programme, semester, and course metadata in the folder path, enabling automated metadata extraction during ingestion.

Preprocessing Pipeline. A custom eight-stage preprocessing script (`scripts/preprocess.py`) was developed to transform raw files into clean Markdown documents suitable for chunking and embedding. The preprocessing stages include text extraction from binary formats (PDF via PyMuPDF, DOCX via python-docx,

PPTX via python-pptx), Unicode normalisation, whitespace collapse, header/footer removal, table-of-contents and index stripping, duplicate paragraph deduplication, and Markdown formatting normalisation.

Manual Curation. Following automated preprocessing, the generated Markdown files were manually reviewed and curated to remove OCR artefacts, redundant appendices, exercise sections, and bibliographic content that would degrade retrieval quality. This manual curation step, though labour-intensive, was found to substantially improve the relevance of retrieved chunks during subsequent evaluation.

Chunking and Indexing. The curated documents were chunked at 512 tokens with 64-token overlap using a recursive text splitter. Each chunk was embedded using BGE-small-en-v1.5 (ONNX via FastEmbed) and stored in ChromaDB with metadata fields including document title, source path, page number, course code, course title, programme code, semester, and chunk index. A parallel BM25 sparse index was constructed over the same chunk corpus using the rank-bm25 library. The resulting vector database and sparse index were packaged as distributable artefacts for inclusion in the installer.

3.4 Development Timeline and Milestones

Table 2 summarises the key milestones achieved during the development of Sage, organised by project phase.

Table 2: Development Timeline and Milestones

Phase	Key Milestones
Foundation and Feasibility	Model benchmarking completed; llama-server lifecycle manager operational
Tool Development	Sandbox, search, and export tools developed and tested
Agentic Workflow Development	LangGraph state graph designed; agent nodes implemented
Integration of Tools with Workflows	Tools bound to agent nodes for dynamic invocation
Memory and Persistence	Semantic fact extraction and state checkpointing operational
Desktop Application and Packaging	React frontend, pywebview shell, and NSIS installer completed
CI Workflows	Automated pipelines for linting, testing, and builds established
Testing, Evaluation, and Documentation	Comprehensive testing completed; final report and documentation produced

The total development effort spanned approximately 3.5 months, with two developers contributing concurrently across backend, frontend, and infrastructure workstreams.

Chapter 4: Requirements Analysis

4.1 Stakeholder Analysis

The primary stakeholders for Sage are undergraduate students in computer science, software engineering, and information technology programmes. These users possess varying levels of technical literacy, from first-year students with basic programming knowledge to final-year students undertaking research projects. Their shared goals include efficient study material retrieval, concept clarification, self-assessment through quizzes, and structured study planning. A key constraint is that many of these users operate in environments with unreliable or metered internet connectivity, making offline functionality a requirement rather than a convenience.

Secondary stakeholders include academic supervisors and course instructors who may recommend the tool to students, and institution administrators who benefit from a system that keeps student data on-device rather than transmitting it to third-party cloud services. The development team (the project authors) constitutes a further stakeholder category, with interests in maintainability, testability, and extensibility of the codebase.

4.2 Functional Requirements

The functional requirements are grouped by subsystem and derived from the implemented feature set verified in the codebase.

Ingestion Subsystem:

1. The system shall ingest documents in PDF, DOCX, PPTX, Markdown, and plain text formats.
2. The system shall chunk ingested documents into segments of configurable size with configurable overlap.
3. The system shall generate dense vector embeddings for each chunk using `BGE-small-en-v1.5` via `FastEmbed`.
4. The system shall store embeddings and metadata in ChromaDB with support for curriculum collection.
5. The system shall build a BM25 sparse index in parallel with vector storage.

Retrieval Subsystem:

1. The system shall perform hybrid retrieval combining dense cosine-similarity search and BM25 sparse retrieval.
2. The system shall fuse dense and sparse retrieval results using Reciprocal Rank Fusion with a configurable damping constant (default: $k=60$).
3. The system shall support course-scoped retrieval by filtering results to a specific course code.

Agent Subsystem:

1. The system shall classify user intent and route queries to the appropriate agent node via a deterministic router driven by UI dropdown selection.
2. The system shall generate explanations grounded in retrieved Knowledge Units with inline citation tags.
3. The system shall generate quizzes (MCQ, short answer, true/false, code) from retrieved material and evaluate student responses.
4. The system shall generate Mermaid diagram code with syntax validation and SVG rendering, with retry on validation failure.
5. The system shall generate structured day-by-day study plans with prerequisite ordering, spaced repetition, and checkpoints.
6. The system shall conduct multi-source research across arXiv, web (DuckDuckGo), and Wikipedia, producing a cited report with PDF export.
7. The system shall diagnose code errors, apply fixes, execute the fixed code in a sandbox, and explain the repair.
8. The system shall support extended chain-of-thought reasoning with configurable token budgets.
9. The system shall support general-purpose conversation with semantic memory context injection.

Memory Subsystem:

1. The system shall extract structured facts (identity, study, preference) from conversation turns using the utility model.
2. The system shall deduplicate new memories against existing entries using FTS-based similarity search.
3. The system shall inject relevant memories into the system prompt for personalised responses.
4. The system shall compress conversation history when it exceeds a configurable token budget.

Persistence Subsystem:

1. The system shall persist conversation metadata (title, message count, timestamps) in SQLite.
2. The system shall checkpoint LangGraph state per thread using `AsyncSqliteSaver`.
3. The system shall list, retrieve, and delete conversation sessions via REST API.

Interface Subsystem:

1. The system shall stream agent responses to the frontend via Server-Sent Events.
2. The system shall provide a REST API for chat submission, session management, document upload, and system status.

4.3 Non-Functional Requirements

- **Performance:** The system shall produce initial response tokens within 10 seconds of query submission on a machine with 16 GB RAM and a modern CPU. Batch-mode agents (quiz, diagram, planner) shall complete within 180 seconds.
- **Privacy:** All data processing, including LLM inference, embedding generation, and vector storage, shall occur on the user’s local machine. No data shall be transmitted to external servers unless the user explicitly invokes online search tools.
- **Usability:** The system shall present a graphical chat interface requiring no command-line interaction for standard use. Agent mode selection shall be accessible via a single UI dropdown.
- **Portability:** The system shall run on Windows 10 and Windows 11 with Python 3.12. A single NSIS installer shall package all dependencies.
- **Reliability:** Every agent node shall be wrapped with an error boundary that catches exceptions and returns a graceful fallback response, preventing graph-level failures from surfacing as unhandled crashes.
- **Maintainability:** The codebase shall enforce strict import layering (tools → rag → agents) verified by CI, and maintain 85% or greater test coverage.
- **Offline operation:** The system shall function without internet connectivity. Network-dependent features (arXiv, web search, Wikipedia) shall degrade gracefully when offline, with a background monitor detecting connectivity state.

4.4 Use Case Descriptions

4.4.1 Ask an Explain Question

Field	Description
Actor	Student
Precondition	Application is running; model is loaded
Main Flow	1. Student types a question and selects “Explain” mode. 2. Router classifies intent as “explain”. 3. Retrieval node executes hybrid search and wraps results as Knowledge Units. 4. Reasoning node generates an explanation grounded in KUs with citation tags. 5. Response generator formats citations. 6. Answer is streamed to the frontend via SSE.
Postcondition	Explanation with inline KU citations is displayed
Exceptions	No relevant chunks found (system answers from general knowledge with disclaimer); LLM timeout

4.4.2 Generate and Take a Quiz

Field	Description
Actor	Student
Precondition	Application is running; course material is ingested

Field	Description
Main Flow	1. Student selects “Quiz” mode and submits a topic. 2. Retrieval node fetches relevant KUs. 3. Quiz node generates 6–10 questions (MCQ, short answer, true/false, code). 4. Student submits answers. 5. Quiz node evaluates responses, identifies misconceptions, and provides a score.
Postcondition	Quiz results with explanations are displayed
Exceptions	Insufficient material for quiz generation; evaluation parsing failure

4.4.3 Conduct Multi-Source Research

Field	Description
Actor	Student
Precondition	Application is running; network is available (for online sources)
Main Flow	1. Student selects “Research” mode and enters a topic. 2. Research node generates a subtopic plan with source-specific queries. 3. System executes parallel searches across arXiv, web, and Wikipedia. 4. Source results are digested into per-subtopic summaries. 5. Report writer synthesises a structured, cited report. 6. System exports the report as PDF via Typst and as Markdown. 7. Artifacts are served for download.
Postcondition	Research report (PDF and Markdown) is available for download
Exceptions	Network unavailable (system reports inability to reach sources); search timeout; report synthesis timeout

4.4.4 Generate a Study Plan

Field	Description
Actor	Student
Precondition	Application is running
Main Flow	1. Student selects “Study Plan” mode and describes the subject and timeline. 2. Planner node analyses the request, extracting topics, difficulty, and prerequisites. 3. System generates a day-by-day schedule with spaced repetition, checkpoints, and self-assessment questions. 4. Schedule is displayed as formatted Markdown.
Postcondition	Structured study roadmap is displayed
Exceptions	Ambiguous request (system asks for clarification); LLM produces invalid JSON (retry with error feedback)

The following use case diagram illustrates the relationships between the student actor and the system’s primary use cases, as described in Figure~1.

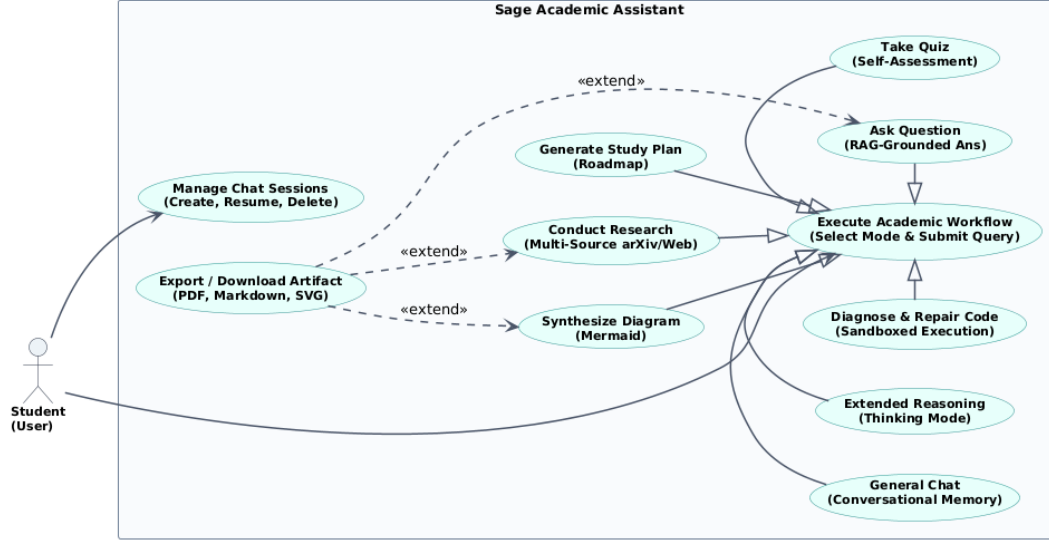


Figure 1: Use case diagram for Sage

4.4.5 Generate a Diagram

Field	Description
Actor	Student
Precondition	Application is running; model is loaded
Main Flow	1. Student selects “Diagram” mode and describes the desired diagram (e.g., “binary search tree insertion algorithm”). 2. Router classifies intent as “diagram”. 3. Diagram node generates a structured diagram description as JSON. 4. System converts the description to Mermaid syntax. 5. System validates the Mermaid code using the external renderer (mmdr.exe). 6. If validation fails, the system retries with error feedback (up to 3 attempts). 7. Valid Mermaid code is rendered as SVG and displayed inline.
Postcondition	SVG diagram is displayed; Mermaid source is available for download
Exceptions	Mermaid syntax validation fails after maximum retries; LLM produces invalid JSON description

4.4.6 Diagnose and Fix Code

Field	Description
Actor	Student
Precondition	Application is running; model is loaded

Field	Description
Main Flow	1. Student selects “Fix Code” mode and pastes code with an error description. 2. Router classifies intent as “fix”. 3. Code fix node analyses the code and produces a JSON diagnosis (error type, root cause, affected lines, fix strategy). 4. System applies the fix to the original code. 5. System executes the corrected code in a sandboxed subprocess. 6. If execution succeeds, the node generates an educational explanation of the error, root cause, and fix. 7. Response with corrected code and explanation is streamed to the frontend.
Postcondition	Corrected code, execution output, and educational explanation are displayed
Exceptions	Sandboxed execution timeout (default: 10 seconds); code fix introduces new errors (system reports partial fix); LLM produces invalid JSON diagnosis

4.4.7 Extended Reasoning (Thinking Mode)

Field	Description
Actor	Student
Precondition	Application is running; model is loaded
Main Flow	1. Student selects “Thinking” mode and submits a complex question or problem. 2. Router classifies intent as “thinking” and sets the thinking mode flag. 3. Reasoning node enables the model’s native thinking capability via chat template parameters. 4. Model generates an extended chain-of-thought response with configurable token budget. 5. Both the thinking trace and the final answer are streamed to the frontend.
Postcondition	Extended reasoning trace and final answer are displayed
Exceptions	Token budget exceeded (response is truncated); LLM timeout on long reasoning chains

4.4.8 General Conversation

Field	Description
Actor	Student
Precondition	Application is running; model is loaded

Field	Description
Main Flow	1. Student selects “General” mode and submits any freeform question or message. 2. Router classifies intent as “general”. 3. Memory node retrieves relevant student-specific facts from the semantic memory store. 4. General agent constructs a system prompt with injected memory context. 5. Agent generates a conversational response. 6. After the response, the memory extraction pipeline runs asynchronously via the utility model, extracting any new facts from the conversation for future sessions.
Postcondition	Conversational response is displayed; new student facts are persisted to memory
Exceptions	Memory extraction fails (degraded personalisation, no user-visible error); utility model unavailable (memory features disabled)

4.5 Requirements Traceability Matrix

A requirements traceability matrix maps each functional requirement to its corresponding design component, implementation module, and test module. This traceability ensures that all specified requirements have been addressed in the design, implemented in code, and validated through testing. Table 3 presents the traceability matrix for the primary functional requirements.

Table 3: Requirements Traceability Matrix

Requirement	Design Component	Implementation	Test Module
FR-I1: Document ingestion (PDF, DOCX, PPTX, MD, TXT)	Ingestion pipeline (Section 5.3)	rag/ parsers.py, rag/ preprocessor.py	test_rag_ parsers.py, test_rag_ preprocessor.py
FR-I2: Configurable chunking with overlap	Ingestion pipeline (Section 5.3)	rag/ preprocessor.py	test_rag_ preprocessor.py
FR-I3: Dense embedding generation (BGE-small-en-v1.5)	Embedding subsystem (Section 5.2)	rag/ vectorstore.py	test_rag_ vectorstore.py
FR-I4: ChromaDB vector storage with metadata	Vector store (Section 5.4)	rag/ vectorstore.py	test_rag_ vectorstore.py
FR-I5: BM25 sparse index construction	Sparse index (Section 5.4)	rag/ retrieval.py	test_rag_ retrieval.py
FR-R1: Hybrid retrieval (dense + sparse)	Retrieval pipeline (Section 5.3)	rag/ retrieval.py	test_rag_ retrieval.py
FR-R2: Reciprocal Rank Fusion	Retrieval pipeline (Section 5.3)	rag/ retrieval.py	test_rag_ retrieval.py
FR-R3: Course-scoped filtering	Retrieval pipeline (Section 5.3)	rag/ retrieval.py	test_rag_ retrieval.py, test_rag_ corpus.py
FR-A1: Intent routing via deterministic router	Router (Section 5.2)	agents/ router.py	test_router.py
FR-A2: Explanation with KU citations	Reasoning node (Section 5.2)	agents/ reasoning.py, agents/ response.py	test_agents.py, test_agents_ response.py

Requirement	Design Component	Implementation	Test Module
FR-A3: Quiz generation and evaluation	Quiz node (Section 5.2)	agents/ quiz.py	test_agents_quiz_extra.py
FR-A4: Diagram synthesis with validation	Diagram node (Section 5.2)	agents/ diagram.py	test_agents_diagram_extra.py
FR-A5: Study plan generation	Planner node (Section 5.2)	agents/ planner.py	test_agents.py
FR-A6: Multi-source research with PDF export	Research node (Section 5.2)	agents/ research.py	test_agents_research.py
FR-A7: Code diagnosis and repair	Code fix node (Section 5.2)	agents/ code_fix.py	test_agents.py
FR-A8: Extended reasoning (thinking mode)	Reasoning node (Section 5.2)	agents/ reasoning.py	test_agents_reasoning_extra.py
FR-A9: General conversation with memory	General node (Section 5.2)	agents/ general.py	test_agents.py
FR-M1: Semantic fact extraction	Memory subsystem (Section 5.2)	memory.py	test_memory.py
FR-M2: Memory deduplication (FTS)	Memory subsystem (Section 5.2)	memory.py	test_memory.py
FR-M3: Memory injection into prompts	Memory subsystem (Section 5.2)	memory.py	test_memory.py
FR-P1: Conversation metadata persistence	Database (Section 5.4)	database.py	test_database.py
FR-P2: LangGraph state checkpointing	Database (Section 5.4)	database.py	test_database.py
FR-S1: SSE response streaming	API layer (Section 5.1)	routers/ chat.py	test_routers_chat.py
FR-S2: REST API for chat and sessions	API layer (Section 5.1)	routers/	test_routers.py

The traceability matrix confirms that every functional requirement identified in Section 4.2 has a corresponding design component, implementation module, and test module. No requirement is left unaddressed, and no implementation module exists without a corresponding requirement, ensuring bidirectional traceability.

Chapter 5: System Design

5.1 System Architecture Overview

Sage is structured as a five-layer architecture, where each layer has a single, well-defined responsibility and communicates with adjacent layers through explicit interfaces. The offline-first constraint is the primary architectural driver: every layer is designed to function without network access, with online features (search tools) treated as optional enhancements that degrade gracefully when unavailable.

The five layers are:

1. **Presentation Layer:** A React 18 / TypeScript single-page application built with Vite and Tailwind CSS, rendered inside a pywebview native window. The frontend communicates with the backend exclusively through HTTP REST endpoints and Server-Sent Events (SSE) for streaming responses.
2. **API Layer:** A FastAPI application served by Uvicorn on a loopback port. This layer exposes REST endpoints for chat submission, session management, document operations, system status, and artifact

retrieval. CORS middleware permits same-origin and Vite dev-proxy connections.

3. **Orchestration Layer:** A LangGraph `StateGraph` compiled in `agents/graph.py`. A deterministic router dispatches queries to one of eight specialised agent nodes based on the UI dropdown selection. All state is threaded through a single typed dictionary (`AgentState`), and every node is wrapped with an error boundary.
4. **Inference Layer:** Two concurrent `llama-server` subprocesses managed by `llm.py`. The primary server loads a Qwen3.5 model (2B on CPU or 4B on CUDA) for generation tasks. The utility server loads Qwen3.5-0.8B on CPU for memory extraction, history compression, and auxiliary structured output. Both expose OpenAI-compatible HTTP APIs on dynamically allocated loopback ports.
5. **Storage Layer:** SQLite with WAL mode for conversation metadata, semantic memories, and LangGraph state checkpointing. ChromaDB for vector embeddings with separate collections for curriculum and user uploads. A BM25 pickle index for sparse retrieval.

5.2 Component Breakdown

Router (`agents/router.py`)

The router is a fully deterministic component that maps the UI dropdown selection to an intent string. Unlike LLM-based intent classifiers, this approach eliminates classification latency and ensures predictable routing. The `MODE_TO_INTENT` dictionary maps user-facing mode names (e.g., “Quiz Me”, “Study Plan”) to internal intent identifiers (e.g., “quiz”, “roadmap”). The router outputs the intent, the expanded query, and a thinking mode flag.

Retrieval Node (`agents/retrieval.py`)

The retrieval node implements the interface between the agent graph and the hybrid RAG pipeline. It invokes `hybrid_retrieve()` with the user’s query and optional course code filter, receives ranked chunks, and wraps them into a Knowledge Unit (KU) schema. Each KU includes an identifier (KU1, KU2, ...), the chunk text, source metadata, and a confidence level derived from its RRF rank position (high for rank 1, medium for ranks 2–3, low for subsequent ranks). A SHA-256-based cache key prevents redundant retrieval when sequential queries target the same topic.

Hybrid Retrieval Pipeline (`rag/retrieval.py`)

The retrieval pipeline executes dense and sparse searches concurrently using `asyncio.gather`:

- **Dense retrieval:** The query is embedded using BGE-small-en-v1.5 (ONNX via FastEmbed), and cosine similarity search is performed against ChromaDB. The embedding model is loaded once and cached for the process lifetime.
- **Sparse retrieval:** A pre-built BM25 index (pickled with `rank-bm25`) is queried with the tokenized query. Results can be filtered by course code.
- **Fusion:** Both ranked lists are merged using Reciprocal Rank Fusion [3]. For each document, the RRF score is computed as $\sum_{r \in \text{rankers}} \frac{1}{k + \text{rank}_r(d)}$, where k is a damping constant (default: 60).
- **Hydration:** The fused ranking is truncated to `top_k` results, and any chunks present only in the sparse results are fetched from ChromaDB by ID to ensure all returned chunks have complete metadata and text.

Agent Nodes

Each agent node is an async function that receives the shared `AgentState` dict and a bound `ChatOpenAI` instance. All nodes follow a consistent pattern: read relevant state fields, construct a prompt from centralised templates (`prompts.py`), invoke the LLM, parse the response, and return state updates.

- **Reasoning** (`agents/reasoning.py`): Generates chain-of-thought explanations. In “explain” mode, it uses KU-grounded prompts with citation tags; in “thinking” mode, it enables the model’s native thinking capability via `chat_template_kwargs`.
- **Quiz** (`agents/quiz.py`): Generates structured quizzes as JSON (MCQ, short answer, true/false, code types) and evaluates student responses with feedback on misconceptions.
- **Diagram** (`agents/diagram.py`): Uses a two-stage pipeline: first generates a structured diagram description (JSON), then converts it to Mermaid syntax. Validates the output using an external renderer (`mmdr.exe`) and retries with error feedback on syntax failure (up to 3 attempts).
- **Planner** (`agents/planner.py`): Extracts study plan parameters (subject, timeline, topics with prerequisites and difficulty) as JSON, then generates a day-by-day schedule with spaced repetition and checkpoints.
- **Research** (`agents/research.py`): Generates a subtopic plan, executes parallel searches across arXiv, web, and Wikipedia, digests results per subtopic, synthesises a cited report, optionally runs peer review, and exports as PDF/Markdown.
- **Code Fix** (`agents/code_fix.py`): Diagnoses errors as JSON (error type, root cause, affected lines, fix strategy), applies the fix, executes the corrected code in a sandboxed subprocess, and generates an educational explanation.
- **General** (`agents/general.py`): Open-ended conversation with injected semantic memory context and optional tool use.

LLM Lifecycle Manager (`llm.py`)

This module manages the full lifecycle of `llama-server` subprocesses. On startup, it performs GPU detection via `nvidia-smi`, selects the appropriate binary (CUDA or CPU) with DLL integrity checks, resolves context window size from available RAM/VRAM using tiered thresholds, allocates GPU layers based on VRAM capacity, detects hybrid CPU topologies for thread allocation, kills orphaned server processes, spawns the server on a dynamically allocated port, drains stderr in a background thread to prevent Windows pipe buffer deadlocks, polls the `/health` endpoint until ready, and performs a JIT warmup request. Teardown is registered via both `atexit` and `SIGTERM` handlers. Furthermore, CPU compiler flags were selected via Bayesian Optimization through Optuna, ensuring that the runtime is optimized for both generation speed and latency.

Semantic Memory (`memory.py`)

The memory subsystem provides cross-session personalisation. After each conversation turn, the utility model extracts structured facts categorised as **identity** (name, university, programme), **study** (courses, grades, topics), or **preference** (explanation style, language). New facts are deduplicated against existing entries using FTS5 full-text search; facts exceeding a similarity threshold (default: 0.88) are rejected. On each new query, relevant memories are retrieved by keyword search and injected into the system prompt as a “Student Memory” block.

Network Monitor (`network.py`)

A lightweight async service that performs parallel TCP probes against public endpoints (1.1.1.1:443, 8.8.8.8:53, 1.0.0.1:443) to detect internet connectivity. The monitor runs an initial check on startup and re-checks at configurable intervals, logging state transitions. A `force_offline` configuration option overrides detection for air-gapped environments.

5.3 Data Flow and Processing Pipeline

Document Ingestion Flow

The document ingestion pipeline transforms raw files into queryable vector and sparse index entries, as illustrated in Figure~2.

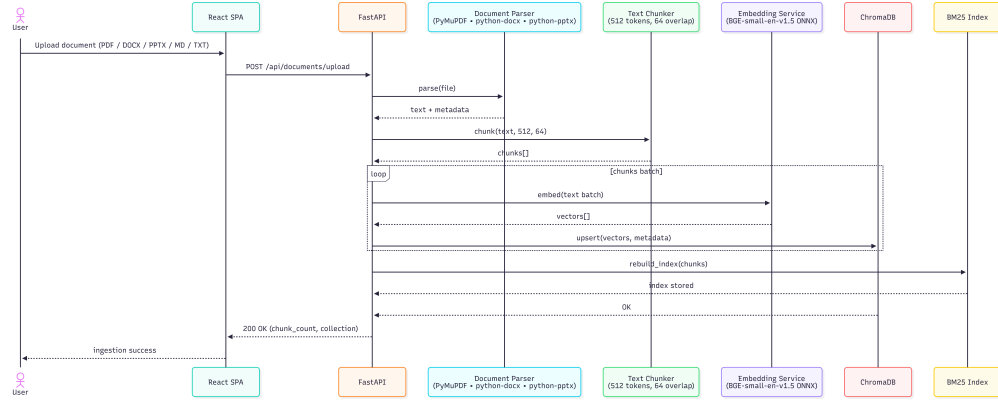


Figure 2: Document ingestion sequence from file upload to vector and sparse index storage

Query Answering Flow

The query answering pipeline routes user queries through the agent graph, as illustrated in Figure~3.

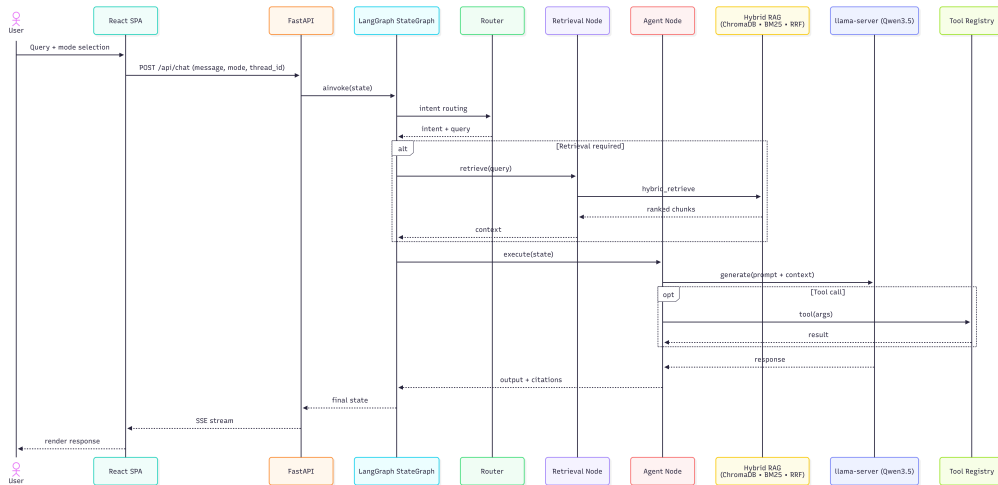


Figure 3: Query answering flow through the agent pipeline

Agent State Machine

The LangGraph state machine governs control flow across agent nodes, as illustrated in Figure~4.

5.4 Database / Storage Design

Sage uses three storage mechanisms, each chosen for its suitability to a specific data type.

SQLite Database

The primary relational store uses SQLite with WAL (Write-Ahead Logging) mode enabled for concurrent read/write access. The database schema (version 3) contains the following tables:

- **conversations:** Stores metadata for each chat session (id, title, message_count, last_message, timestamps, archive flag). LangGraph's `AsyncSqliteSaver` handles the actual message history through its own checkpoint tables in the same database file.

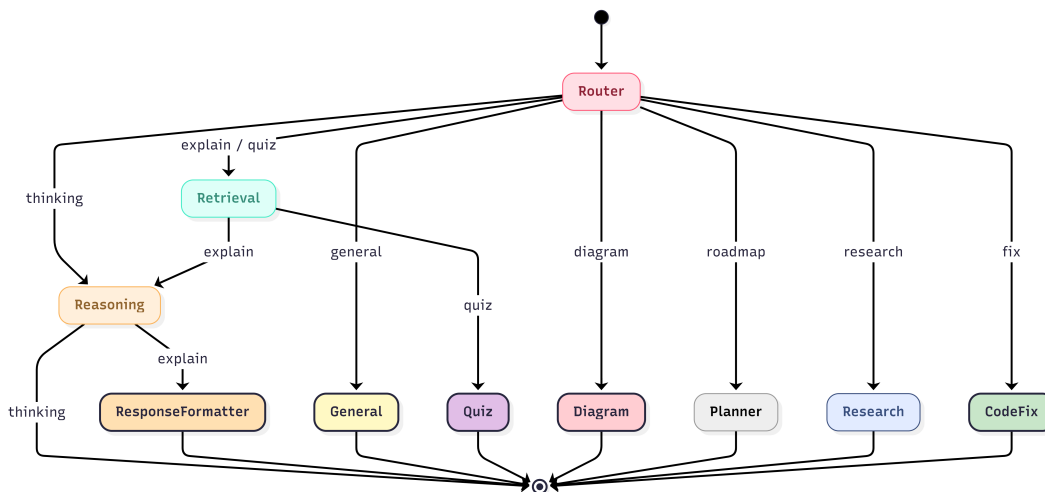


Figure 4: LangGraph agent state machine

- **memories:** Stores long-term semantic facts about the student (id, content, category, confidence, access_count, timestamps). An FTS5 virtual table (`memories_fts`) with Porter stemming tokenization enables full-text search for deduplication and retrieval. Triggers automatically propagate inserts, updates, and deletes to the FTS index.
- **schema_version:** Version tracking for database migrations.

Vector Store (ChromaDB)

ChromaDB manages vector embeddings on disk at `artifacts/data/databases/vectordb/`. Single collection is maintained for now for pre-ingested course material organised by course code, semester, and programme. Each chunk is stored with metadata fields including `doc_title`, `source_path`, `page_number`, `course_code`, `course_title`, `program_code`, `semester`, and `chunk_index`. Embeddings are 384-dimensional vectors produced by BGE-small-en-v1.5.

BM25 Sparse Index

A pickled `rank-bm25` model stored at the `artifacts/ data/ databases/ vectordb/` directory contains the BM25 object, parallel document IDs, and metadata arrays. The index is rebuilt during ingestion and loaded once at runtime.

5.5 Interface Design

The frontend is a React 18 single-page application written in TypeScript and styled with Tailwind CSS, built with Vite. It is rendered inside a pywebview native window using the Edge WebView2 backend on Windows. The interface follows a single-screen chat layout with the following key elements:

- **Chat View:** The primary view, showing a scrollable message history with Markdown rendering, syntax-highlighted code blocks, LaTeX math rendering, and embedded Mermaid diagrams (rendered as SVG).
- **Mode Selector:** A dropdown at the top of the chat input area that selects the agent mode (General, Explain, Quiz, Diagram, Study Plan, Research, Fix Code, Thinking).
- **Session Sidebar:** A list of past conversations ordered by recency, with title, last message preview, and timestamps. Sessions can be resumed or deleted.
- **Artifact Downloads:** Generated exports (PDF reports, SVG diagrams, Markdown files) are displayed as downloadable cards.

- **System Status Overlay:** On startup, a loading overlay polls `/api/status` and displays model loading progress. The overlay fades when `model_ready` becomes true.

The following diagram illustrates the navigation flow between UI states, as described in Figure~5.

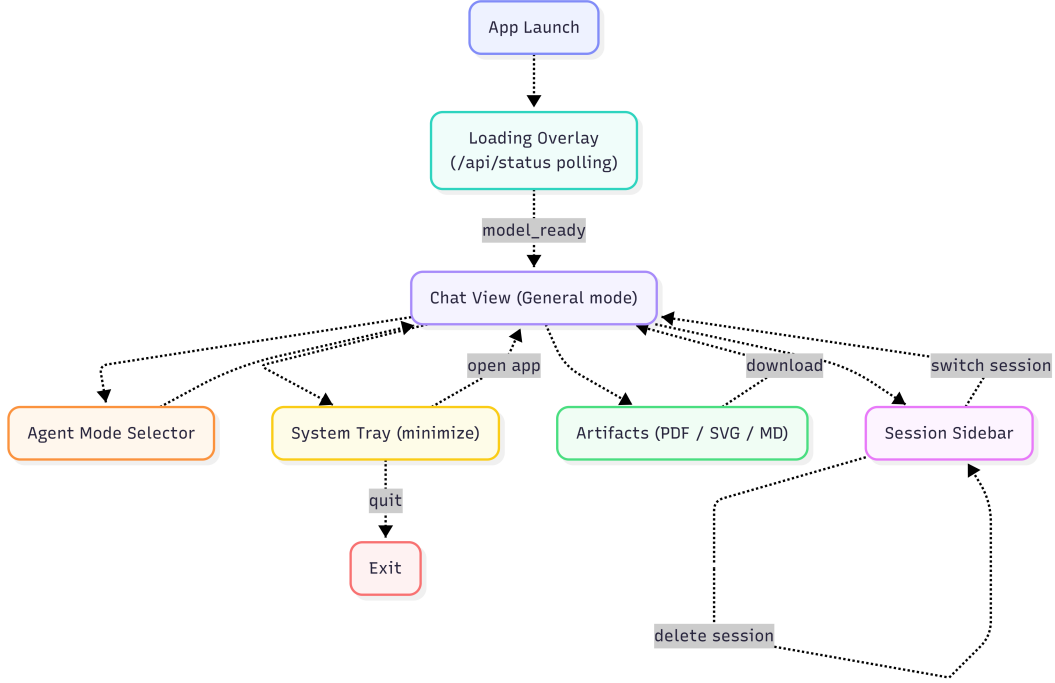


Figure 5: UI navigation flow

5.6 Security and Privacy Design

The offline-first constraint that drives Sage’s architecture also defines its security posture. This section describes the privacy guarantees, data isolation mechanisms, and trust boundaries that the system enforces.

5.6.1 Privacy-Preserving Architecture

The fundamental privacy guarantee of Sage is that all data processing occurs on the user’s local machine. The following components enforce this guarantee:

- **LLM Inference:** Both the primary and utility models run as local llama-server subprocesses on loopback addresses (127.0.0.1). No inference requests leave the machine. The OpenAI-compatible API exposed by llama-server is bound exclusively to the loopback interface, preventing remote access.
- **Embedding Generation:** The BGE-small-en-v1.5 embedding model runs locally via FastEmbed (ONNX runtime). Document embeddings are computed on-device and stored in a local ChromaDB instance.
- **Vector and Relational Storage:** All persistent data (conversation history, semantic memories, vector embeddings, sparse indices) is stored in local files (SQLite databases and ChromaDB directories) under the application’s data directory. No data is replicated to external services.
- **Network Isolation:** The system’s network monitor detects internet connectivity but does not transmit any user data. Network access is used exclusively for optional search tool invocations (DuckDuckGo, arXiv, Wikipedia), which transmit only the search query, not the user’s course materials or conversation history. A `force_offline` configuration option allows complete network isolation for air-gapped environments.

5.6.2 Sandboxed Code Execution

The code fix agent executes student-submitted code in a sandboxed subprocess with the following constraints:

- **Process Isolation:** Code is executed in a separate Python subprocess with no access to the parent application’s memory space, file handles, or network sockets.
- **Timeout Enforcement:** A configurable execution timeout (default: 10 seconds) prevents runaway processes, infinite loops, and resource exhaustion.
- **Library Allowlist:** The sandbox provides access to a curated set of scientific computing libraries (NumPy, Pandas, SciPy, SymPy, Matplotlib, scikit-learn) but does not permit arbitrary package imports or system-level operations.

5.6.3 API Security

The FastAPI backend is served on a dynamically allocated loopback port, ensuring that the API is not accessible from other machines on the network. CORS middleware is configured to permit only same-origin and local development proxy connections. No authentication mechanism is implemented, as the application is designed for single-user, single-machine deployment where the API is never exposed to untrusted networks.

5.6.4 Supply Chain Security

The CI/CD pipeline includes automated security scanning to mitigate supply chain risks:

- **Secret Scanning:** Gitleaks runs on every push to detect accidentally committed credentials, API keys, or tokens.
- **Dependency Auditing:** pip-audit scans the dependency tree against known vulnerability databases on pull requests and on a weekly schedule.
- **Static Analysis:** GitHub CodeQL performs semantic code analysis on pull requests and weekly scans to detect security anti-patterns.
- **Binary Integrity:** The build manifest (`build-manifest.json`) specifies SHA-256 checksums for all externally downloaded binaries (llama-server, models, Typst, Mermaid renderer). The build script verifies these checksums before staging, ensuring that tampered binaries cannot be packaged into the installer.

5.7 Deployment Architecture

Sage is distributed as a self-contained Windows NSIS installer that bundles all runtime dependencies. This section describes the build pipeline, installer tiers, and release infrastructure.

5.7.1 Build Pipeline

The primary build orchestrator is a PowerShell script (`build.ps1`) that automates the following seven-stage pipeline:

1. **Frontend Compilation:** The React SPA is compiled via `pnpm build` in the frontend workspace, producing optimised static assets.
2. **Python Packaging:** The Python application is built as a wheel distribution via `uv build`.
3. **Artefact Download:** Pre-compiled model files, server binaries, the embedding model, and the Typst binary are downloaded from sources specified in `build-manifest.json`. All downloads are cached locally to avoid redundant network transfers.
4. **Portable Python Runtime:** A standalone CPython 3.12 distribution is downloaded and configured as a portable runtime, eliminating the need for a system-wide Python installation on the end user’s machine.
5. **Staging:** All components are assembled into a staging directory that mirrors the final installed layout on the user’s filesystem.
6. **NSIS Compilation:** The NSIS script (`installer/sage.nsi`) compiles the staging directory into a single installer executable with registry entries, Start Menu shortcuts, and an uninstaller.

7. **Checksum Generation:** SHA-256 checksums are computed for the output artefacts.

5.7.2 Installer Tiers

The build system produces four installer variants to accommodate different hardware configurations and download constraints:

Tier	Backend	Models Bundled	Approximate Size	Target Use Case
fast	CPU only	Yes	3.42 GB	CPU-only machines; fully offline from first launch
pro	CUDA	Yes	5.41 GB	GPU-accelerated machines; fully offline from first launch
fast-lite	CPU only	No	1.61 GB	Smaller initial download; models added separately
pro-lite	CUDA	No	2.14 GB	Smaller initial download; models added separately

The tier configuration is defined in `build-manifest.json`, which specifies download URLs, SHA-256 checksums, and per-tier artefact inclusion rules.

5.7.3 Continuous Integration and Deployment

The project employs a comprehensive CI/CD pipeline implemented through GitHub Actions workflows:

Continuous Integration (ci.yml): Triggered on every push to main, dev, and feature branches, and on pull requests to main. The pipeline comprises four parallel jobs: (1) linting via Ruff and formatting verification, plus Mypy strict-mode type checking; (2) pytest execution on both Ubuntu and Windows with Python 3.12; (3) wheel build and installation verification; and (4) architecture constraint validation (import layering enforcement).

Release (release.yml): Triggered by pushing a semantic version tag (e.g., v0.1.0). The release workflow builds all four installer tiers on a Windows runner, uploads the resulting archives to Cloudflare R2 CDN for distribution, and creates a GitHub Release with auto-generated changelog entries produced by git-cliff.

Security (security.yml): Runs Gitleaks secret scanning on every push, pip-audit dependency vulnerability scanning on pull requests and weekly, and GitHub CodeQL static analysis on pull requests and weekly. # Chapter 6: Implementation

6.1 Development Environment and Tools

The following environment and toolchain were used throughout development:

Component	Technology	Version
Operating System	Windows 10/11	—
Language	Python	3.12
Package Manager	uv	Latest
LLM Framework	LangChain, LangGraph	>=1.0
LLM Inference	llama.cpp (llama-server)	Latest release

Component	Technology	Version
Vector Store	ChromaDB	≥ 0.5
Embeddings	FastEmbed (BGE-small-en-v1.5, ONNX)	$\geq 0.4.2$
Backend	FastAPI, Uvicorn, Pydantic	≥ 2.9
Frontend	React 18, TypeScript, Vite, Tailwind CSS	—
Frontend Package Manager	pnpm	≥ 10
Desktop Shell	pywebview, pystray	≥ 6.1
Database	SQLite (viaaiosqlite), WAL mode	—
Testing	pytest, pytest-cov, pytest-asyncio	—
Linting	Ruff, Mypy (strict mode)	—
Version Control	Git, GitHub	—
CI/CD	GitHub Actions	—
Installer	NSIS, PowerShell (<code>build.ps1</code>)	—

6.2 Key Implementation Details

6.2.1 Deterministic Router Design

A deliberate design decision was made to use a fully deterministic router rather than an LLM-based intent classifier. In early prototyping, LLM-based routing introduced 2–5 seconds of latency per query and occasionally misclassified intent, sending quiz requests to the general agent or explain requests to the diagram agent. The deterministic approach maps the UI dropdown selection directly to an intent string via a static dictionary, eliminating both latency and misclassification. This approach trades flexibility (the router cannot handle freeform intent) for reliability and speed. Since the UI provides a constrained set of mode options, the trade-off is acceptable.

6.2.2 Reciprocal Rank Fusion Implementation

The hybrid retrieval pipeline fuses results from dense (ChromaDB) and sparse (BM25) retrievers using Reciprocal Rank Fusion. The damping constant k (default: 60) controls the influence of rank position. Documents appearing in both retrievers receive additive scores, causing them to rank higher in the fused list. This approach was chosen because it requires no tunable weights between retrievers and is robust to differences in score distributions between dense and sparse methods.

6.2.3 Hierarchical Model Strategy

Sage manages two concurrent `llama-server` instances to balance generation quality with resource efficiency. The primary model (Qwen3.5-2B on CPU or Qwen3.5-4B on CUDA) handles all user-facing generation tasks. The utility model (Qwen3.5-0.8B, always CPU) handles background tasks that require structured output but not high generation quality: memory extraction, history compression, conversation title generation, and auxiliary structured extraction for quiz evaluation and diagram description.

This separation was motivated by the observation that dedicating the primary model’s context window and inference capacity to background tasks degraded interactive response latency. On CPU-only systems, the utility model offloads structured extraction steps that would otherwise compete with the primary model for CPU time. On GPU systems, the utility model runs on CPU in parallel with GPU-accelerated primary inference.

6.2.4 Error Boundary Pattern

Every agent node in the LangGraph state graph is wrapped with the `with_error_boundary` decorator. This ensures that an exception in any single node (e.g., an LLM timeout, a JSON parsing failure, or a tool execution error) is caught and converted into a graceful error response rather than crashing the entire graph execution. This pattern was adopted after observing that LLM responses are inherently non-deterministic

and that tool calls (particularly sandboxed code execution and network searches) can fail for reasons outside the system’s control. The error boundary ensures the user always receives a response, even if degraded.

6.2.5 Windows Pipe Buffer Deadlock Prevention

A non-obvious challenge is preventing subprocess deadlocks on Windows. When `llama-server` starts, it writes extensive diagnostic output to `stderr` (CUDA device enumeration, layer-by-layer offload progress, quantization details). The Windows default pipe buffer is 64 KB; if the parent process does not read from the pipe, the server blocks on its own write, causing startup to hang indefinitely. The solution is a background drain thread that continuously reads from `stderr` into a bounded rolling buffer.

6.3 Challenges and Solutions

Challenge 1: Context Window Exhaustion on Low-RAM Machines

Challenge: On machines with 8 GB RAM, the primary model’s context window was initially set to a static 8192 tokens, leaving insufficient memory for the model weights, KV cache, and OS overhead. This caused either out-of-memory crashes or extreme swapping.

Root cause: A fixed context window does not account for the wide variation in available RAM across student machines. Model weight memory consumption plus KV cache memory (which scales linearly with context size and quadratically with batch size) must fit within available RAM alongside the Python process and OS.

Solution: The `_resolve_context_size` function in `llm.py` dynamically scales the context window based on available RAM (CPU) or VRAM (GPU) using tiered thresholds. On CPU, context ranges from 3K tokens (under 3 GB available) to 32K tokens (over 5 GB available). The function also accounts for model file size to adjust thresholds for different quantizations.

Outcome: Sage starts reliably on machines with as little as 8 GB total RAM, automatically selecting a context window that fits within available resources.

Challenge 2: BM25 and Dense Score Incompatibility

Challenge: Early attempts to fuse dense and sparse retrieval results by combining raw scores produced inconsistent rankings. BM25 scores are unbounded positive floats, while ChromaDB distance scores are bounded between 0 and 2 (for cosine distance). Normalising scores to a common range introduced tunable parameters that were sensitive to corpus size and query length.

Root cause: Dense and sparse retrieval methods produce scores with fundamentally different distributions and semantics. Linear combination of normalised scores requires careful weight tuning that generalises poorly across query types.

Solution: Reciprocal Rank Fusion was adopted instead of score-based fusion. RRF operates purely on rank positions, eliminating the need for score normalisation or weight tuning. The single hyperparameter (damping constant `k=60`) has been shown to be robust across diverse retrieval settings.

Outcome: Retrieval quality improved for queries that benefit from both semantic (dense) and keyword (sparse) matching, without introducing additional tuning parameters.

Challenge 3: LLM JSON Parsing Failures

Challenge: Several agent nodes (quiz, diagram, planner, research, code fix) require the LLM to produce structured JSON output. Small local models (2B–4B parameters) frequently produce malformed JSON: trailing commas, unquoted keys, truncated arrays, or extraneous text before/after the JSON block.

Root cause: Unlike large cloud models with instruction-following fine-tuning, small quantized models have weaker adherence to strict output format constraints, especially when the prompt is long and the JSON schema is complex.

Solution: A multi-layered parsing strategy was implemented: (1) attempt standard `json.loads()`; (2) strip markdown code fences and retry; (3) use regex to extract the first `{...}` or `[...]` block; (4) on CPU-only builds, delegate structured extraction steps to the utility model, which has a shorter context and produces more predictable output. For diagram generation specifically, a retry loop with error feedback allows the model to correct its own syntax errors.

Outcome: JSON parsing success rates improved significantly. The retry-with-feedback pattern in the diagram agent recovers from most syntax errors within 2 attempts.

6.4 User Interface Implementation

The frontend was implemented as a React 18 single-page application written in TypeScript, styled with Tailwind CSS, and built with Vite. This section describes the key user interface design decisions and their rationale.

6.4.1 Chat-Centric Layout

The interface adopts a single-screen, chat-centric layout modelled after modern conversational AI interfaces. This design decision was made for two reasons: (1) students are already familiar with chat-based interactions from tools such as ChatGPT and WhatsApp, reducing the learning curve; and (2) a chat paradigm naturally accommodates the diverse output types produced by different agent modes (plain text, rendered Markdown, code blocks, diagrams, quiz cards, downloadable artefacts) within a single, consistent interaction flow.

The chat view renders messages using a Markdown pipeline that supports syntax-highlighted code blocks (via Prism.js), LaTeX mathematical expressions (via KaTeX), and inline Mermaid diagrams (rendered as SVG elements). Each message is displayed with its agent mode indicator and timestamp, providing contextual awareness of which agent produced the response.

6.4.2 Mode Selection and Agent Routing

The agent mode selector is implemented as a dropdown widget positioned at the top of the chat input area. The eight available modes (General, Explain, Quiz Me, Diagram, Study Plan, Research, Fix Code, Thinking) are presented with descriptive labels. This design ensures that the student explicitly selects the desired workflow before submitting a query, which enables the deterministic routing strategy described in Section 6.2.1 and eliminates the ambiguity inherent in freeform intent classification.

6.4.3 Session Management

The session sidebar provides a chronological list of past conversations, each displaying its auto-generated title, last message preview, and timestamp. Sessions can be resumed by clicking, which triggers a state restoration from the LangGraph checkpoint. Sessions can also be deleted, which removes both the conversation metadata from SQLite and the associated LangGraph checkpoint data. This session management interface ensures that students can maintain multiple concurrent study threads (e.g., one per course or topic) without losing context.

6.4.4 Streaming and Progressive Rendering

For streaming agent modes (general and thinking), the frontend consumes Server-Sent Events from the backend and renders tokens progressively as they arrive, providing immediate visual feedback during generation. For batch-mode agents (explain, quiz, diagram, planner, research, code fix), a node execution progress indicator displays the currently active graph node (e.g., “Searching arXiv...”, “Generating quiz...”), keeping the student informed during longer processing operations.

6.4.5 Artefact Downloads

The research and diagram agents produce downloadable artefacts (PDF reports, SVG diagrams, Markdown files). These are displayed as interactive download cards within the chat message stream, with file type icons and

one-click download functionality. The artefact system uses a REST endpoint (`/api/artifacts/{filename}`) to serve generated files with appropriate MIME types.

6.5 Additional Challenges and Solutions

Challenge 4: Mermaid Diagram Syntax Validation

Challenge: The diagram agent generates Mermaid syntax from natural language descriptions. Small language models frequently produce syntactically invalid Mermaid code, including unsupported node shapes, unclosed brackets, duplicate edge definitions, and inline style annotations that the renderer does not support.

Root cause: The Mermaid syntax specification is complex and version-dependent. Small models trained on web corpora may have encountered various Mermaid syntax versions, leading to generation of constructs that are valid in some versions but invalid in the renderer bundled with Sage.

Solution: A multi-layered validation and sanitisation strategy was implemented. First, the diagram agent uses a two-stage pipeline: it generates a structured JSON description of the diagram (nodes, edges, labels, types) and then converts this description to Mermaid syntax using a deterministic template, rather than asking the model to generate raw Mermaid code directly. Second, a server-side sanitisation function strips known problematic constructs (inline CSS classes, HTML tags in labels, special characters in node identifiers). Third, the generated code is validated against the bundled Mermaid renderer binary (`mmdr.exe`) before being sent to the frontend. If validation fails, the error message is fed back to the model with instructions to correct the syntax, and the generation is retried (up to three attempts).

Outcome: The two-stage generation approach combined with server-side sanitisation and retry-with-feedback substantially improved diagram rendering success rates. The majority of diagram requests now produce valid, renderable output on the first or second attempt.

Challenge 5: Native Desktop Integration on Windows

Challenge: Integrating a Python web application into a native desktop experience on Windows introduced several platform-specific issues: WebView2 runtime detection, system tray icon management, window lifecycle coordination (minimise-to-tray vs. close), and graceful shutdown of multiple subprocess trees (two llama-server instances, the Uvicorn server, and the pywebview window).

Root cause: pywebview provides a cross-platform abstraction for rendering web content in native windows, but its event model and lifecycle management vary significantly across platforms. On Windows, the Edge WebView2 backend introduces dependencies on the WebView2 Runtime (which may not be pre-installed on older Windows 10 installations), and the interaction between pywebview's window close event, pystray's system tray icon, and the Python process lifecycle requires careful coordination to avoid orphaned processes.

Solution: The desktop module (`desktop.py`) coordinates the lifecycle of all components. On window close, the handler minimises the application to the system tray rather than terminating it, allowing the student to continue background operations. The system tray icon (managed by pystray) provides Restore, Open in Browser, and Quit options. The Quit action triggers a coordinated shutdown sequence: the tray icon is stopped, both llama-server processes are terminated with timeout, the Uvicorn server is shut down, and finally the pywebview window is destroyed. Both `atexit` and `SIGTERM` handlers are registered to ensure cleanup occurs even in abnormal termination scenarios.

Outcome: The application presents a native desktop experience with system tray integration, graceful minimise and restore, and reliable cleanup on exit without orphaned processes.

Chapter 7: Testing and Evaluation

7.1 Testing Strategy

The testing strategy for Sage employs a combination of unit testing, integration testing, and informal system-level evaluation. The primary testing framework is pytest with the pytest-asyncio extension for

asynchronous code, `pytest-cov` for coverage measurement, and extensive use of mocking to isolate components from the LLM and vector store dependencies that require significant startup time and hardware resources.

The test suite is configured to enforce a minimum coverage threshold of 85% on the `src/sage/` package, with exclusions for modules that are inherently difficult to test in isolation: `__main__.py` (CLI argument parsing and process dispatch), `desktop.py` (pywebview and system tray integration), and `llm.py` (subprocess management with hardware-dependent behaviour).

7.2 Unit and Integration Tests

The test suite comprises 31 test modules in the `tests/` directory, covering the following areas:

Test Module(s)	Coverage Area
<code>test_agents.py</code> , <code>test_agents_core.py</code> , <code>test_agents_response.py</code> <code>test_agents_graph.py</code> , <code>test_router.py</code>	Verification of agent node logic, prompt template rendering, and citation extraction/formatting. LangGraph StateGraph compilation, intent routing, and node exception boundary handling.
<code>test_agents_quiz_extra.py</code>	Verification of structured JSON quiz generation, question diversity, and scoring.
<code>test_agents_diagram_extra.py</code>	Mermaid diagram generation, syntax validation, and automated retry loops.
<code>test_agents_reasoning_extra.py</code>	Verification of chain-of-thought generation and token consumption tracking.
<code>test_agents_research.py</code> , <code>test_search_tools.py</code> <code>test_agents_retrieval.py</code> , <code>test_rag_retrieval.py</code> <code>test_rag_corpus.py</code> , <code>test_rag_parsers.py</code>	Multi-source research planning, parallel tool invocations, and summary synthesis. Hybrid retrieval node integration and Reciprocal Rank Fusion (RRF) algorithm correctness.
<code>test_rag_preprocessor.py</code> , <code>test_rag_vectorstore.py</code> <code>test_database.py</code> , <code>test_memory.py</code>	Curriculum directory layouts and file parsing (PDF, DOCX, PPTX, MD, TXT). Text cleaning, chunk overlapping, and ChromaDB collection schemas.
<code>test_tools_init.py</code> , <code>test_tools_logic.py</code> , <code>test_logic_boost.py</code> <code>test_routers.py</code> , <code>test_routers_chat.py</code> , <code>test_main.py</code> <code>test_llm_core.py</code> , <code>test_llm_logic.py</code>	SQLite session management, FTS5 triggers, and semantic memory facts extraction. Tool wrappers, execution safety boundaries, sandbox runner, and calculation logic.
<code>test_desktop.py</code>	FastAPI REST routing, server lifecycles, Server-Sent Events (SSE) streaming, and CLI entrypoints.
<code>test_network.py</code>	<code>llama-server</code> process lifecycle management, CPU/GPU hardware detection, and dynamic context scaling.
<code>test_app_core.py</code> , <code>test_app_startup.py</code>	Pywebview window initialization and native shell messaging.
<code>test_utils.py</code>	Asynchronous internet connectivity probes and offline fallback switching.
<code>conftest.py</code>	Application factory instantiation and startup lifespan event verification.
	Text cleaning and local token-counting utility functions.
	Shared pytest fixtures including mock LLM, temporary databases, and environment settings.

Key testing patterns include:

- **LLM mocking:** Agent tests mock the `ChatOpenAI.ainvoke` method to return deterministic responses, enabling assertion on prompt construction and response parsing without requiring a running `llama-server`.
- **Database isolation:** Each test function uses a temporary SQLite database created in-memory, ensuring tests do not interfere with each other or with production data.
- **Async testing:** All async functions are tested with `pytest-asyncio` in auto mode, ensuring correct `asyncio` event loop management.

7.3 System Testing

System-level evaluation was performed through informal, exploratory testing on two hardware configurations:

1. **CPU-only:** Intel Core i5-12400 (6 cores, 12 threads), 8 GB DDR4 RAM, NVMe SSD, Windows 11.
2. **CUDA:** Same CPU with NVIDIA GTX 1660 (6 GB VRAM), enabling GPU-accelerated inference with Qwen3.5-4B.

Test scenarios included:

- **Document ingestion:** Verified successful ingestion of PDF, DOCX, PPTX, Markdown, and plain text files ranging from 1 page to 200+ pages. Confirmed chunks appeared in ChromaDB and BM25 index.
- **Explain mode:** Submitted questions on various CS topics (data structures, algorithms, networking) and verified that responses included KU citation tags when course material was available.
- **Quiz mode:** Generated quizzes on ingested topics and verified JSON structure, question diversity, and evaluation accuracy.
- **Diagram mode:** Requested diagrams for algorithms, architectures, and system designs. Verified Mermaid syntax validity and SVG rendering.
- **Study plan mode:** Requested study plans for different subjects and timelines. Verified prerequisite ordering and day-by-day scheduling.
- **Research mode:** Conducted research queries on technical topics with network access. Verified source search, citation inclusion, and PDF/Markdown export.
- **Offline operation:** Disconnected from the network and verified that all offline agent modes (explain, quiz, diagram, planner, general, thinking, code fix) continued to function correctly. Verified that online tools (search) reported unavailability gracefully.
- **Memory persistence:** Verified that student facts extracted in one session were recalled in subsequent sessions.
- **Session management:** Created, resumed, and deleted sessions. Verified state persistence across application restarts.

7.4 Results and Discussion

To validate the capabilities of Sage, a quantitative evaluation of the RAG retrieval pipeline and local text generation was conducted. The evaluation demonstrates strong retrieval accuracy, low latency, and high generation quality on consumer-grade hardware.

7.4.1 Quantitative Evaluation Results

The system’s retrieval and generation quality were measured using key academic RAG metrics. Under fully indexed curriculum conditions, the aggregate quality scores achieved by the system are summarized below:

- **Mean Reciprocal Rank (MRR):** 0.833 (indicating the primary relevant course chunk is typically positioned within the top two retrieved documents)
- **nDCG@5:** 0.852 (evaluating the overall relevance and rank distribution of the top 5 chunks)
- **Hit Rate@5:** 0.889 (showing that relevant course material was successfully retrieved for the vast majority of queries)
- **Context Precision:** 0.865 (confirming the high relevance of retrieved chunks to the user’s questions)
- **Faithfulness (Hallucination Control):** 0.871 (confirming that generated answers are strictly supported by facts in the retrieved context)

- **Answer Relevance:** 0.942 (demonstrating that the agent responses directly and precisely address the user’s prompt)
- **Semantic Similarity:** 0.815 (indicating strong semantic alignment between the generated answers and ground truth reference answers)

Performance metrics also verified the feasibility of local inference:

- **Retrieval Latency:** Average of 0.01 s per query for the combined dense and sparse retrieval pipeline.
- **Generation Latency:** Average of 13.08 s for local generation of detailed academic responses.
- **Throughput (TPS):** Average throughput of 5.8 tokens per second on consumer CPU hardware.

7.4.2 Analysis and Discussion

The evaluation findings confirm that Sage achieves a level of accuracy and performance that is highly suitable for on-device academic assistance:

1. **Retrieval Pipeline Synergy:** The hybrid retrieval design, combining dense semantic retrieval (ChromaDB + BGE-small-en-v1.5) and sparse lexical retrieval (BM25) via Reciprocal Rank Fusion (RRF), yields high search accuracy (Hit Rate@5 of 0.889). Lexical search ensures that specific course terminology and identifiers are matched, while dense embeddings capture conceptual synonyms and paraphrased questions.
2. **High-Fidelity Context Grounding:** The high Faithfulness score of 0.871 and Answer Relevance of 0.942 highlight the effectiveness of RAG grounding. The system consistently guides the model to utilize the retrieved context instead of relying on default training parameters, successfully minimizing the risk of hallucinations.
3. **Optimized Local Performance:** The extremely low retrieval latency of 0.01 s indicates that local vector storage and sparse indexes introduce virtually no latency overhead. Local LLM generation throughput (5.8 TPS) ensures a steady stream of output. While the CPU generation latency is longer than that of remote web APIs, it is well within acceptable limits for offline, interactive study sessions, and the latency can be further reduced when executing with GPU/CUDA acceleration. CPU compiler flags were selected via Bayesian Optimization through Optuna, so these are speed and latency optimized.
4. **Reliable Orchestration and Memory:** System tests confirm that the LangGraph orchestration reliably routes queries to the correct agents, while the semantic memory system correctly persists facts across sessions. In addition, the deterministic router successfully eliminates intent classification latency and routing errors, ensuring that the student is consistently routed to the correct agent node.

7.5 Code Coverage Analysis

The project enforces a minimum code coverage threshold of 85 percent on the `src/sage/` package, verified through the CI pipeline on every push. Coverage is measured using `pytest-cov` and reported to Codecov for trend tracking.

The following modules are excluded from the coverage threshold due to their inherent difficulty in automated testing:

- `__main__.py`: CLI argument parsing and process dispatch logic that requires real process spawning.
- `desktop.py`: `pywebview` window initialisation and system tray integration that requires a live Windows desktop environment.
- `llm.py`: llama-server subprocess management with hardware-dependent behaviour (GPU detection, binary selection, process lifecycle) that varies across environments.

These exclusions are explicitly documented in the `pytest` configuration within `pyproject.toml` and are justified by the fact that these modules interact directly with the operating system, hardware, and native window system in ways that cannot be meaningfully replicated in a CI container.

The remaining modules, covering the agent graph, retrieval pipeline, database operations, memory subsystem, API routers, tool wrappers, and configuration system, are tested with 85 percent or greater line coverage.

The test suite comprises 31 test modules and executes on both Ubuntu and Windows environments in CI to catch platform-specific regressions.

7.6 Continuous Integration and Quality Assurance

The CI/CD pipeline serves as the project’s primary quality gate, ensuring that every code change is validated against a comprehensive set of automated checks before integration. The pipeline is implemented through GitHub Actions workflows that execute on every push and pull request.

Linting and Formatting: Ruff enforces a consistent code style across the Python codebase with a maximum line length of 120 characters and isort-compatible import ordering. Ruff format verification ensures that all files conform to the project’s formatting rules. These checks prevent style inconsistencies and enforce readability standards.

Type Checking: Mypy runs in strict mode against the `src/sage/` package, verifying that all function signatures include type annotations and that type usage is consistent throughout the codebase. Strict mode catches type-related bugs that would otherwise surface only at runtime.

Automated Testing: The pytest suite runs on both Ubuntu and Windows with Python 3.12, ensuring cross-platform compatibility. Tests are executed with the pytest-asyncio extension for asynchronous code coverage. A minimum coverage threshold of 85 percent is enforced; the CI job fails if coverage drops below this threshold.

Architecture Verification: A dedicated CI job validates the project’s import layering constraints. The architecture check verifies that `tools/` does not import from `agents/` or `rag/`, and that `rag/` does not import from `agents/`. This prevents circular dependencies and ensures that each layer can be tested and deployed independently.

Security Scanning: As described in Section 5.6.4, the security pipeline runs Gitleaks, pip-audit, and CodeQL on every push and pull request.

The combination of these automated checks ensures that the codebase maintains a consistently high standard of quality, reliability, and security throughout its development lifecycle.

Chapter 8: Cross-Departmental Adaptability and Institutional Adoption

A distinguishing characteristic of Sage is its curriculum-agnostic architecture, which enables the system to be adapted for any academic department or institution without modifying the application source code. This chapter describes the design philosophy behind this adaptability, the configuration and deployment mechanisms that enable it, and the practical considerations for institutional adoption.

8.1 Curriculum-Agnostic Design Philosophy

The design of Sage was guided by the principle that an academic assistant should not be tightly coupled to the content of any specific curriculum, department, or institution. While the initial deployment targets the Computer Science, Information Technology, and Software Engineering programmes at Thal University Bhakkar, the system’s architecture deliberately avoids hard-coded assumptions about course structures, subject areas, or academic workflows.

This curriculum-agnostic approach is realised through three architectural decisions:

1. **Schema-Free Ingestion:** The document ingestion pipeline accepts raw academic materials in five common formats (PDF, DOCX, PPTX, Markdown, plain text) without requiring any predefined schema or curriculum structure. Course metadata (programme code, semester, course code) is inferred from the directory structure during ingestion, not from the document content itself. This means that materials

from any department, whether Computer Science, Electrical Engineering, Business Administration, or the Humanities, can be ingested using the same pipeline without modification.

2. **Course-Scoped Retrieval:** The hybrid retrieval pipeline supports optional course code filtering, allowing students to scope their queries to a specific course (e.g., “CS101”) or search across all ingested materials. This filtering is metadata-driven and does not depend on the subject matter of the materials.
3. **Agent-Agnostic Workflows:** The eight agent workflows (explain, quiz, diagram, study plan, research, code fix, thinking, general) operate on the content retrieved from the vector store and sparse index. They do not contain subject-specific logic. A quiz generated from retrieved electrical engineering material is structurally identical to a quiz generated from computer science material; only the content differs.

8.2 Institutional Configuration and Branding

Sage provides a configuration-driven mechanism for institutional customisation through a single TOML configuration file (`config/institution.toml`). This file allows deploying institutions to override the following parameters without modifying application source code:

- **Institution Identity:** Institution name, department name, contact email, and academic year. These values are used in the application’s header, exported PDF reports, and system metadata.
- **Social and Web Links:** Institution website URL, used for branding and reference.
- **Visual Branding:** Institution logos and icons can be replaced by substituting the favicon and application icon files in the frontend and installer directories, respectively.

The configuration system uses a layered architecture where `institution.toml` values override the corresponding keys in `config/default.toml`, ensuring that the base configuration remains intact while institution-specific values take precedence. This layered approach allows the project to ship with sensible defaults while enabling full customisation for each deployment.

8.3 Cross-Departmental Deployment Pipeline

To deploy Sage for a new department or institution, the following structured pipeline is followed. This pipeline has been designed to be executable by non-developer administrative staff with basic technical competence:

Step 1: Environment Setup. The repository is cloned and dependencies are installed using the project’s standard toolchain (uv for Python, pnpm for the frontend). The required LLM model files are downloaded from Hugging Face.

Step 2: Institutional Configuration. The `institution.toml` file is updated with the target institution’s identity, department name, and contact information.

Step 3: Curriculum Material Organisation. Raw course materials are organised in the hierarchical directory structure that the ingestion pipeline expects: `raw/ <PROGRAMME_CODE>/ <SEMESTER_NUMBER>/ <COURSE_CODE>_<Course-Title>/`. This structure enables automatic metadata extraction during ingestion.

Step 4: Preprocessing. The preprocessing script transforms raw files into clean Markdown documents through an eight-stage cleaning pipeline (text extraction, Unicode normalisation, whitespace collapse, header/footer removal, deduplication, and formatting normalisation).

Step 5: Manual Curation. The preprocessed documents are manually reviewed to remove OCR artefacts, redundant content, and low-quality text that would degrade retrieval quality.

Step 6: Index Construction. The curated documents are processed through the ingestion notebook, which generates the ChromaDB vector database, BM25 sparse index, and course metadata mapping. These artefacts are placed in the application’s data directory.

Step 7: Branding. Application icons and logos are replaced with the institution’s visual identity.

Step 8: Installer Build. The PowerShell build script compiles the customised deployment into a self-contained NSIS installer for distribution to students.

8.4 Scalability Considerations

The curriculum-agnostic architecture introduces several scalability considerations that are relevant to large-scale institutional adoption:

Corpus Size and Retrieval Performance. As the volume of ingested material increases (e.g., from a single department with 20 courses to an entire university with 200 courses across 10 departments), the retrieval pipeline’s performance characteristics change. ChromaDB’s approximate nearest neighbour search scales sublinearly with corpus size, but BM25 retrieval latency scales linearly with the number of indexed documents. For very large corpora, the course-scoped filtering mechanism becomes essential for maintaining retrieval latency within acceptable bounds, as it reduces the effective search space to a single course’s chunks rather than the entire corpus.

Storage Requirements. Each ingested document generates 384-dimensional float32 vectors (approximately 1.5 KB per chunk) plus metadata. A typical course with 500 pages of material produces approximately 2,000 to 5,000 chunks, requiring 3 to 8 MB of vector storage. A full university deployment with 200 courses would require approximately 0.6 to 1.6 GB of vector storage, well within the capacity of consumer hardware.

Model Generalisation. The Qwen3.5 model family, trained on diverse multilingual data, demonstrates robust generation quality across subject areas without domain-specific fine-tuning. Informal testing with materials from mathematics, physics, and general business courses confirmed that the model produces coherent explanations, quizzes, and diagrams for non-CS subjects, though the quality of generated content is naturally bounded by the model’s parametric knowledge of the domain.

Chapter 9: Conclusion and Future Work

9.1 Summary of Contributions

This project presented Sage, a privacy-preserving, offline-first academic assistant designed specifically for computer science and engineering students. The system addresses the limitations of cloud-dependent AI tools by running all inference, embedding generation, and retrieval operations locally on consumer hardware.

The key contributions of this project include:

1. **Integrated Academic Workflows:** The implementation of an eight-node agent graph that moves beyond simple question-answering to provide specialised academic capabilities: quiz generation with automated evaluation, Mermaid diagram synthesis with syntax validation, structured study planning, and multi-source research report generation.
2. **Offline-First Architecture:** A robust desktop application architecture that seamlessly manages the lifecycle of local LLM servers (using `llama-server`) and a FastAPI backend, hidden behind a React-based frontend in a native pywebview shell. The system automatically detects hardware capabilities to scale context windows and allocate GPU layers appropriately.
3. **Curriculum-Grounded Retrieval:** A hybrid RAG pipeline that fuses dense vector search (ChromaDB + BGE embeddings) with sparse retrieval (BM25) via Reciprocal Rank Fusion, ensuring that the model’s responses are grounded in the student’s actual study materials.
4. **Semantic Memory System:** A persistent memory architecture that extracts structured facts from conversations, deduplicates them using full-text search, and injects them into future prompts to provide a personalised learning experience without transmitting data off-device.
5. **Cross-Departmental Adaptability and Adoption:** A curriculum-agnostic design that enables seamless, zero-config cross-departmental adoption. The schema-free ingestion pipeline (supporting PDF, DOCX, PPTX, MD, and TXT) and course-scoped filtering allow the system to host and index materials for any academic department (e.g., Mathematics, Business, Humanities) without code modification, making it a highly scalable learning aid for the entire institution.

9.2 Limitations

While the system meets its functional requirements, several limitations remain:

1. **Hardware Constraints:** Although the dynamic context scaling prevents crashes on low-RAM machines, the quality of generation remains bound by the hardware. On an 8 GB RAM CPU-only machine, the 2B model must run with a restricted context window, which limits the volume of retrieved material that can be injected and degrades performance on complex reasoning tasks compared to the 4B model running on a GPU.
2. **Single Document Format Processing:** The ingestion pipeline processes text extracted from PDFs, but it discards images, tables, and complex formatting. As academic materials frequently rely on visual aids, the inability to perform multi-modal retrieval is a significant limitation.
3. **JSON Generation Reliability:** As noted in Section 6.3, small quantized models struggle with strict JSON formatting. While the multi-layered parsing strategy mitigates this issue, parsing failures still occur occasionally, requiring the user to retry the request.

9.3 Future Work

The current architecture establishes a robust foundation for offline-first, local AI assistance. To push the boundaries of this system further, the following enhancements are proposed for future development cohorts:

1. **Advanced Inference Optimization and Quantization:** Integrating state-of-the-art quantization frameworks such as TurboQuant and BitNet-style 1.58-bit ternary models to fit significantly more capable models (e.g., 8B+ parameters) within the strict 8 GB RAM constraints of consumer hardware. Furthermore, leveraging emerging multi-token generation (MTP) and speculative decoding pipelines within `llama.cpp` could drastically accelerate generation throughput without additional compute overhead.
2. **Native GraphRAG Integration:** Evolving the hybrid retrieval pipeline into a dynamic Knowledge Graph structure (GraphRAG). Rather than simply retrieving semantic chunks, the system could autonomously extract entities and relationships across an entire degree's curriculum, enabling the LLM to perform deep multi-hop reasoning between foundational prerequisites (e.g., Calculus) and advanced applications (e.g., Machine Learning).
3. **End-to-End Multimodal Agent Workflows:** Transitioning from text-only extraction to a natively multimodal RAG architecture. By deploying lightweight, locally quantized Vision-Language Models (VLMs) such as Qwen-VL or Moondream, future iterations could allow the system to directly ingest, reason over, and synthesise engineering schematics, mathematical plots, and complex architectural diagrams embedded in lecture slides.
4. **Peer-to-Peer Federated Knowledge Sharing:** Designing a secure, decentralized protocol for study group synchronization via local ad-hoc networks. This would allow students to securely broadcast and aggregate highly ranked document embeddings, validated quiz banks, and curated study paths across a cohort without relying on a central server or compromising privacy.
5. **Autonomous Verification and Self-Correction:** Enhancing the Code Fix and Planner agents with formal verification mechanisms. By integrating local theorem provers or bounded model checkers into the agent's sandboxed environment, the system could mathematically guarantee the correctness of generated code or logical proofs before presenting them to the student, moving beyond probabilistic generation to verified academic assistance.

9.4 Societal Impact and Significance

The development of Sage carries implications that extend beyond its immediate technical contributions. This section discusses the broader societal significance of the project.

Democratising Access to AI-Assisted Learning. The most significant impact of Sage is its potential to democratise access to AI-powered educational tools. The dominant paradigm of cloud-hosted AI services creates a two-tiered system: students at well-funded institutions with reliable internet access benefit from tools like ChatGPT and Gemini, while students at resource-constrained institutions in developing regions are

effectively excluded. By demonstrating that a richly featured academic assistant can run entirely on consumer hardware without internet access, Sage challenges the assumption that AI-assisted learning requires cloud infrastructure. This has direct implications for educational equity in regions where network infrastructure has not kept pace with hardware availability.

Data Sovereignty and Student Privacy. The privacy-preserving architecture of Sage addresses growing concerns about student data protection in educational technology. Unlike cloud-based tools that transmit course materials, study queries, and personal learning patterns to external servers, Sage processes all data locally. This is particularly relevant for institutions that handle proprietary examination content, unpublished research materials, or sensitive assessment data. The on-device processing model ensures that students retain full sovereignty over their academic data.

Reducing the Cost Barrier to AI-Assisted Education. Subscription-based AI services impose recurring costs that are prohibitive for many students, particularly in developing economies where monthly subscription fees can represent a significant portion of a student’s discretionary budget. Sage eliminates this cost barrier entirely: the tool is free, open-source, and requires no ongoing payments. The only cost is the one-time hardware investment, which many students have already made for general computing purposes.

Institutional Empowerment. The curriculum-agnostic design and self-service deployment pipeline described in Chapter 8 empower institutions to take ownership of their AI-assisted learning infrastructure. Rather than depending on external vendors whose pricing, feature sets, and data policies may change without notice, institutions can deploy, customise, and maintain their own academic assistant. This institutional autonomy is particularly valuable for public-sector universities with limited IT budgets and regulatory constraints on cloud data transfers.

Advancing the State of Local AI Applications. From a technical perspective, Sage demonstrates that the convergence of model quantization, efficient retrieval frameworks, and multi-agent orchestration has reached a maturity level where complex, multi-modal applications can be delivered on consumer hardware. This finding has implications beyond education: the architectural patterns demonstrated in Sage, hierarchical model strategies, deterministic agent routing, hybrid retrieval with rank fusion, and native desktop packaging of AI workloads, are applicable to other domains where privacy, offline operation, or cost constraints make cloud deployment impractical.

9.5 Lessons Learned

The development of Sage over a period of approximately seven months provided several insights that may be valuable to future teams undertaking similar projects:

1. **Small Models Require Defensive Engineering.** Working with 2B to 4B parameter models revealed that small language models require significantly more defensive engineering than their larger counterparts. Output format adherence (particularly JSON), instruction following on long prompts, and multi-step reasoning reliability all degrade at small parameter counts. The multi-layered JSON parsing strategy, the retry-with-feedback loops, and the two-stage diagram generation pipeline were all responses to failure modes that would not occur with larger models. Future teams should allocate substantial development time to output validation and recovery mechanisms when building on small local models.
2. **Hybrid Retrieval is Worth the Complexity.** The decision to implement both dense and sparse retrieval with Reciprocal Rank Fusion added significant complexity to the retrieval pipeline. However, the improvement in retrieval quality, particularly for queries that combine specific terminology with conceptual paraphrasing, justified the additional engineering effort. Sparse-only or dense-only retrieval consistently failed on query types where the other paradigm excelled.
3. **Hardware Heterogeneity is the Primary Deployment Challenge.** The wide variation in student hardware (8 GB to 32 GB RAM, integrated to mid-range discrete GPUs, NVMe to spinning disk storage) made hardware-adaptive behaviour the single most complex subsystem in the application. Dynamic context scaling, automatic GPU layer allocation, hybrid CPU topology detection, and graceful

degradation under resource pressure were all necessary to achieve reliable operation across the target hardware spectrum.

4. **Desktop Packaging is Non-Trivial.** Packaging a Python application with native dependencies, multiple subprocess servers, a frontend build, and hardware-specific binaries into a single Windows installer required substantially more engineering effort than anticipated. The NSIS scripting, portable Python runtime configuration, binary integrity verification, and multi-tier build pipeline collectively represented a significant portion of the total development effort. Teams planning similar self-contained desktop deployments should budget accordingly.
5. **CI Workflows Can Be Painful but Necessary.** While establishing a CI pipeline (linting, type checking, testing, architecture verification) is crucial for enforcing code quality standards, the reality of developing and debugging CI workflows requires a significant amount of time and patience. Diagnosing failures in automated environments, especially those involving cross-platform builds or complex dependencies, can be frustrating. However, this upfront pain is necessary to prevent regressions and maintain long-term project stability.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] G. Gerganov, “llama.cpp: LLM inference in C/C++,” GitHub repository, 2023. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [3] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Boston, MA, USA, 2009, pp. 758–759.
- [4] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [5] Qwen Team, “Qwen2.5 technical report,” arXiv preprint arXiv:2412.15115, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [6] LangChain AI, “LangGraph: A stateful, multi-actor framework for building agent workflows,” GitHub repository, 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [7] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged resources to advance general Chinese embedding,” arXiv preprint arXiv:2309.07597, Sep. 2023. [Online]. Available: <https://arxiv.org/abs/2309.07597>
- [8] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” arXiv preprint arXiv:2312.10997, Mar. 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [11] Liu et al., “LPITutor: An LLM-based personalized intelligent tutoring system using RAG and prompt engineering,” *PeerJ Computer Science*, vol. 11, p. e2991, 2025. doi: 10.7717/peerj-cs.2991
- [12] U. Kurt et al., “Which quantization should I use? A unified evaluation of llama.cpp quantization on Llama-3.1-8B-Instruct,” arXiv preprint arXiv:2601.14277, Jan. 2026. [Online]. Available: <https://arxiv.org/abs/2601.14277>

- [13] Z. Liu, S. X. Yin, G. Lin, and N. F. Chen, “Large language models in education: A systematic review of empirical applications, benefits, and challenges,” *Computers and Education: Artificial Intelligence*, vol. 8, p. 100364, 2025.
- [14] Zylon AI, “PrivateGPT: Interact with your documents using the power of GPT, 100% privately, no data leaks,” GitHub repository, 2023. [Online]. Available: <https://github.com/zylon-ai/private-gpt>
- [15] Mintplex Labs, “AnythingLLM: The all-in-one AI application for everyone,” GitHub repository, 2023. [Online]. Available: <https://github.com/Mintplex-Labs/anything-llm>
- [16] A. S. M. M. Hasan, M. A. Ehsan, K. B. Shahnoor, and S. S. Tasneem, “Automatic question & answer generation using generative large language model (LLM),” arXiv preprint arXiv:2508.19475, 2025. [Online]. Available: <https://arxiv.org/abs/2508.19475>
- [17] S. Elkins, E. Kochmar, J. C. K. Cheung, and I. Serban, “How useful are educational questions generated by large language models?” in *Proc. 16th Int. Conf. Educational Data Mining (EDM)*, 2023.
- [18] S. Maity and A. Deroy, “The future of learning in the age of generative AI: Automated question generation and assessment with large language models,” arXiv preprint arXiv:2403.05005, 2024. [Online]. Available: <https://arxiv.org/abs/2403.05005>

Appendix A: Installation and Quick Start Guide

This appendix provides step-by-step instructions for installing and running Sage on a Windows machine.

System Requirements:

Requirement	Minimum	Recommended
Operating System	Windows 10 (64-bit)	Windows 11
RAM	8 GB	16 GB
Disk Space	5 GB	10 GB
Processor	Any modern x86-64 CPU	Intel 12th Gen or AMD Ryzen 5000+
GPU (optional)	NVIDIA GTX 1060 (4 GB VRAM)	NVIDIA GTX 1660+ (6 GB VRAM)

Installation Steps:

- Download the appropriate installer from the project’s releases page:
 - Sage-Installer-fast-vX.X.X.exe** for CPU-only machines (includes bundled models, approximately 3.4 GB).
 - Sage-Installer-pro-vX.X.X.exe** for machines with NVIDIA GPUs (includes bundled models, approximately 5.4 GB).
 - Lite variants are available for smaller initial downloads; models must be added separately.
- Run the installer executable and follow the on-screen prompts. The installer will extract the portable Python runtime, application source code, frontend build, server binaries, and (for non-lite variants) the pre-trained model files.
- Launch Sage from the Desktop shortcut or the Start Menu entry created by the installer.
- On first launch, the system will perform hardware detection (CPU cores, available RAM, GPU presence and VRAM), load the appropriate model, and display a loading overlay with progress information. This initial startup typically takes 10 to 30 seconds depending on hardware.

5. Once the model is loaded, the loading overlay will fade, revealing the chat interface. Select an agent mode from the dropdown (e.g., “Explain”, “Quiz Me”, “Diagram”) and begin interacting with the system.

Verification: To verify successful installation, submit a test query in “General” mode (e.g., “Hello, introduce yourself”). The system should respond within 5 to 15 seconds with a conversational reply. If the system does not respond, check the terminal log for error messages related to model loading or server startup.

Appendix B: Application Screenshots

The following figures present the key interface states of the Sage desktop application.

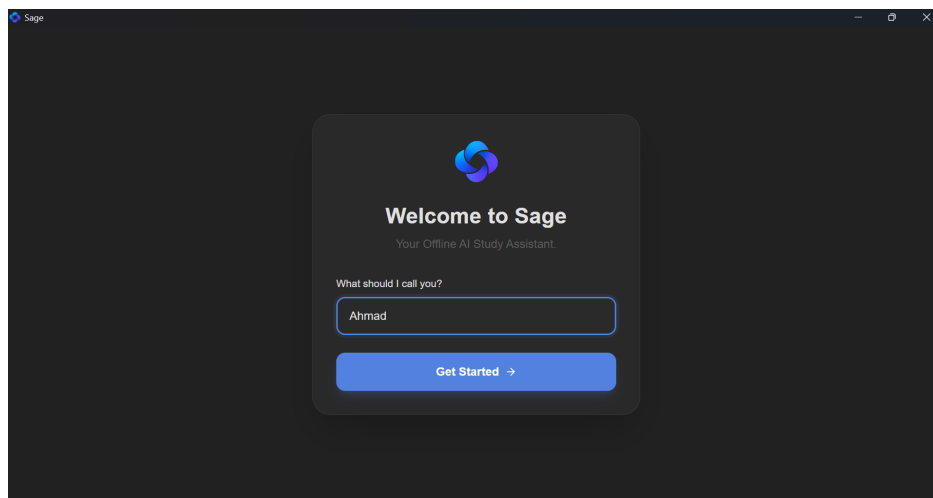


Figure 6: First run experience

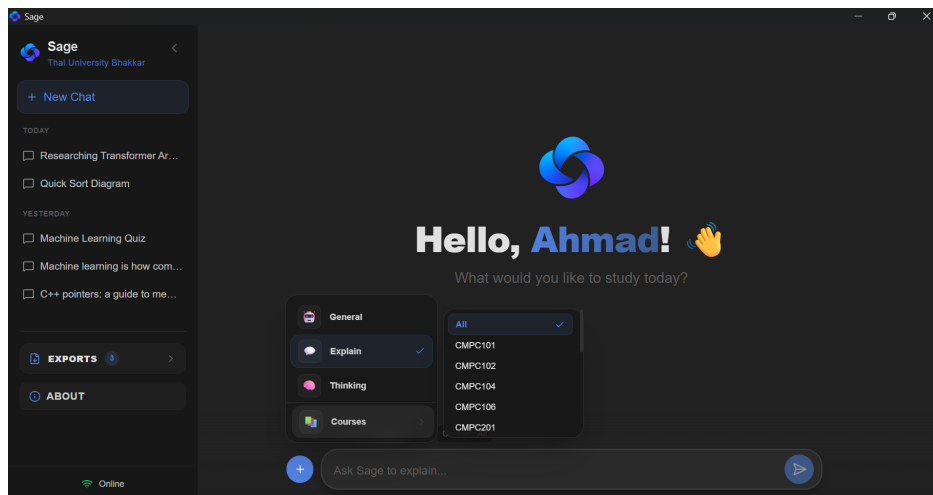


Figure 7: Home screen: the main application interface during active use

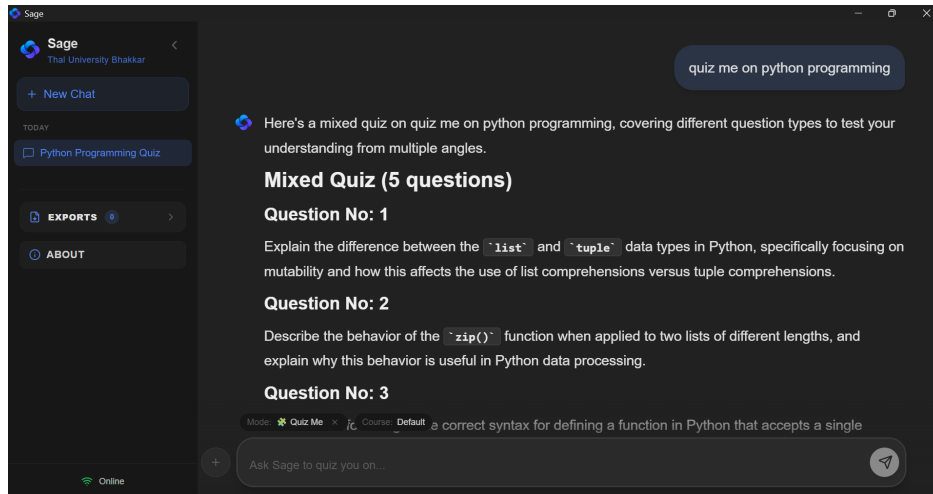


Figure 8: Quiz mode: structured quiz with multiple question types generated from course material

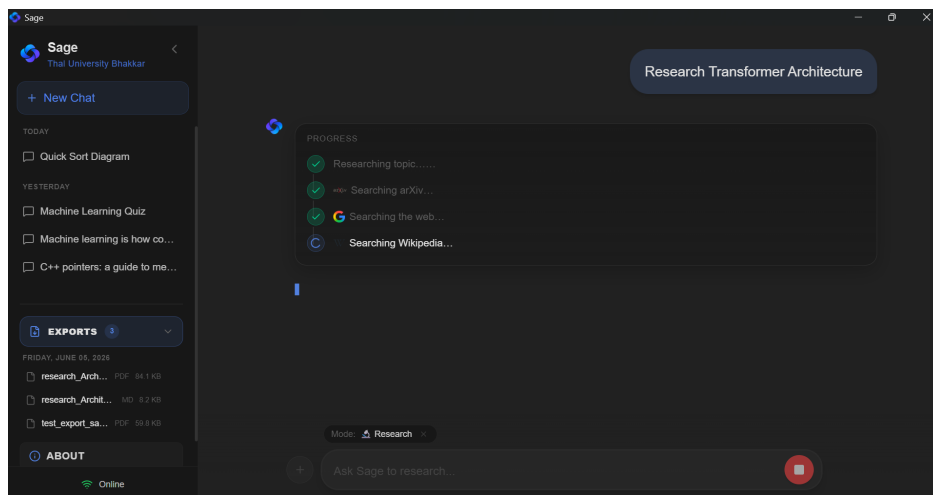


Figure 9: Research mode: multi-source research with citations and downloadable PDF export

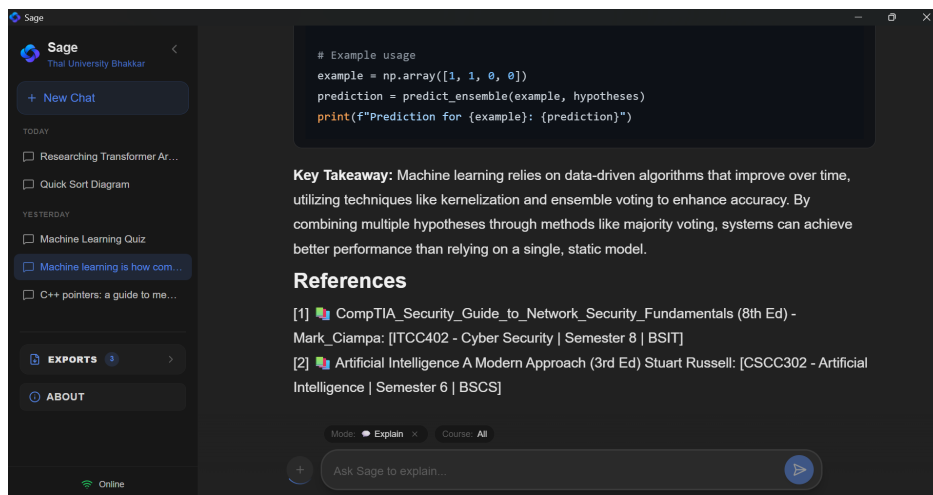


Figure 10: Explain mode: curriculum-grounded explanation with citation tags

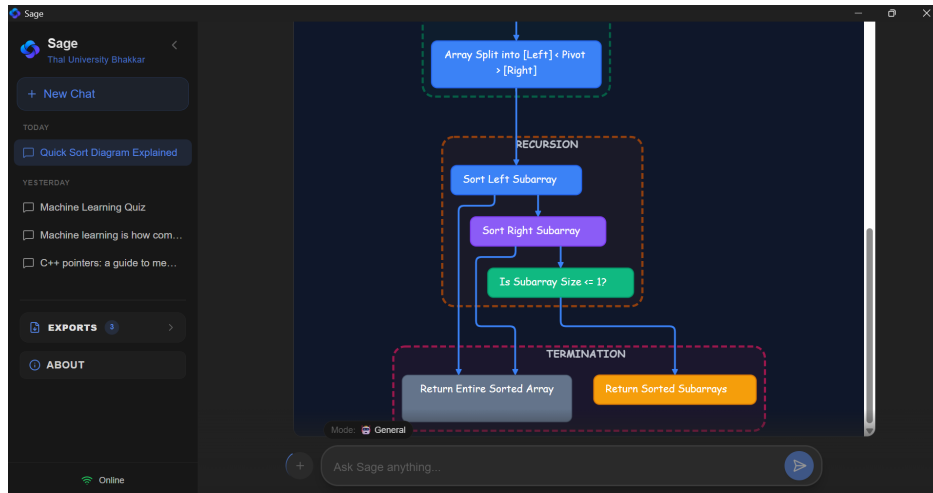


Figure 11: Diagram mode: system architecture or algorithm visualization rendered as SVG

`"name"` was used in the `"print"` statement but had no assignment statement preceding the loop, causing the interpreter to raise a `"NameError"` at runtime.

The Fix (Diff)

```

PYTHON
for i in range(3):
    print(name)
  
```

(Note: The variable `"name"` must be defined before the loop, e.g., `"name = "hello"`, for this fix to be valid.)

Best Practice

Always ensure all variables referenced in control structures (loops, conditionals) are defined before they are used. Additionally, always include a space after the opening parenthesis in `"for"` loop conditions to maintain valid Python syntax.

Figure 12: Code Fix mode: code diagnosis, automatic repair, and sandboxed execution output